oracle :-
=======

1 DB fundamentals
2 SQL
3 PL/SQL ----

Data :-
=====

=> Data are raw facts i.e. facts about people,places,events etc

ex :- 100,sachin,40,hyderabad

Information :-
=========

=> organized or processed data is called informtion

ex :- empid ename age city
100 sachin 40 hyderabad

Database :-
=========

=> db is a organized collection of interrelated data , for ex
a bank db stores data related to customers,trans,loans etc
and univ db stores data related to course,students,faculty etc.

Types of Databases :-
================

1 OLTP DB (online transaction processing)
2 OLAP DB (online analytical processing)

=> oltp db stores day-to-day transactions and olap is used for analysis
=> oltp stores current data and olap stores historical data

applications----------OLTP DB----------OLAP DB----------ANALYZE

html	oracle	oracle	power bi
js	mysql	postgresql	tableau

java	postgresql	sql server	python
.net	mongoDB		
android		sql server	

=> day-to-day opertions on db includes

C create
R read
U update
D delete

DBMS :-
=======

=> DBMS stands for Database Management System
=> DBMS is a software used to work with database
=> using DBMS software users can create,read,update,delete data in db
=> DBMS is an interface between user and database.

USER----------DBMS----------DB

Types of DBMS (Evolution of DBMS) :-
==========

1960	fms (file mgmt system)
1970	hdbms (hierarchical dbms) ndbms (network dbms)
1980	rdbms (relational dbms)
1990	ordbms (object relational dbms)

RDBMS :-
=========

=> Relational Database Management System
=> rdbms concepts introduced by Edgar Frank Codd
=> E.F.CODD introduced 12 rules called codd rules
=> a db software that supports all 12 rules called perfect rdbms software

Information rule :-
==========

=> according to information rule data must be organized in tables i.e.
rows and columns

cust

custid	name	addr	=> columns / fields / attributes
100	sachin	hyd	
101	rahul	blr	
102	vijay	del	=> row / record / tuple

DB => collection of tables
TABLE => collection of rows & cols
ROW => collection of field values
COLUMN => collection of values of one field

=> every table must contain primary key to uniquely identify the rows

ex :- accno,empid,aadharno,panno,voterid

=> one table related to another table using foreign key

projects

projid	name	duration	cost	client
1000	DBS	3	400	DBS Bank
1001	LFT	2	150	Lufthansa Airlines

emp

empid	ename	job	sal	projid	(fk)
100	K	SE	30K		1000
101	S	SSE	80K	1000	
102	D	SE	40K		1001

=> To establish relationship between two tables , take primary key
of one table and add it to another table as foreign key

RDBMS features :-
==========

1 easy to access & manipulate data
2 less redundency (duplication of data)
3 more security
4 gurantees data quality
5 supports data sharing

6 supports transctions (ACID properties)

RDBMS softwares :-
==========

SQL Databases :-
==========

ORACLE
MYSQL
SQL SERVER
POSTGRESQL
DB2
SQLite

NoSQL Databases :-
==========

MongoDB
cassandra
cosmos DB
redis

21-sep-26

ORDBMS :-
==========

=> object relational dbms
=> it is the combination of rdbms and oops

ordbms = rdbms + oops (reusability)

=> rdbms doesn't support reusability but ordbms supports reusability

RDBMS :-
==========

cust
cid cname hno street city state pin

emp
empid ename hno street city state pin

ORDBMS :-
==========

addr
hno street city state pin

cust
cid cname addr
hno street city state pin

emp
empid ename addr
hno street city state pin

ORDBMS softwares :- -
===============

ORACLE
MYSQL
SQL SERVER
POSTGRESQL

what is db ?
what is dbms ?
what is rdbms ?
what is ordbms ?

vehicle	dbms
2 wheeler	rdbms
tvs scooter	oracle

==========

ORACLE :-
=========

=> oracle is basically a rdbms software and also supports features of ordbms
=> organizations uses oracle software to manage db i.e. to work with db
=> oracle software is used for db development and administration.

Development	Administration
creating tables	installation of oracle
creating views	creating database
creating synonyms	creating userids
creating sequences	db backup & restore
creating materialized views	db export & import
creating indexes	performance tuning
creating procedures	
creating functions	
creating triggers	
writing queries	

versions :-
========

2,3,4,5,6,7,8i,9i,10g,11g,12c,18c,19c,21c,23ai,26ai

i => internet
g => grid
c => cloud
ai => aritificial intelligence

=> before 8 version oracle supports desktop applications and from 8 onwards oracle supports internet applications , so suffix "i" is added.

=> from 10g onwards oracle db can be accessed through multiple servers called grid , the advantage of grid is it improves availability i.e. db is always available to customers.

=> from 12c onwards oracle db can be deployed in

1 on premises
2 on cloud

=> In " on premises " db is deployed in the server managed by client.

=> In " on cloud " db is deployed in the server managed by cloud service provider for ex amazon.

=> oracle 26ai supports vector databases which are based on semantic search, so from 26ai onwards ai tools can also use oracle database.

CLIENT / SERVER Architecture :-
==========

1 server
2 client

server :-
=======

=> server is a system where oracle software is installed
=> inside the server oracle manages

1 DB
2 INSTANCE

=> DB is created in hard disk and acts as permanent storage
=> INSTANCE is created in ram and acts as temporary storage
=> data temporarly stored in instance and after saving it is copied to db

client :-
=======

=> client is also a system from where users can

1 connects to server
2 submit requests to server
3 receive response from server

client tools :-
=========

1 sqlplus (cui)
2 sql developer (gui)

user----------sqlplus----------oracle----------db

SQL :-
======

=> SQL stands for structured query language
=> a language used to communicate with oracle
=> user communicates with oracle by sending commands called queries

=> a query is a command / instruction / question given to oracle to perform operations on db
=> SQL is originally introduced by IBM and initial name of this language was SEQUEL
and later it is renamed to SQL.
=> SQL is common to all relational db softwares

user-----sqlplus----------sql----------oracle----------db

user----mysqlworkbench---------sql----------mysql----------db

user-----ssms----------sql----------sql server----------db

user------pgadmin----------sql----------postgresql---------db

=> based on operations over db SQL is categorized into following sub-languages

DDL (Data Definition Lang)
DML (Data Manipulation Lang)
DQL (Data Query Lang)
TCL (Transaction Control Lang)
DCL (Data Control Lang)

SQL

DDL	DML	DQL	TCL	DCL	
create	insert	select	commit	grant	
alter	update		rollback	revoke	
drop	delete		savepoint		
truncate	merge				
rename	insert all				
flashback					
purge					

DATA & DATA DEFINITION :-
==========

empid	ename	sal	data definition / metadata
100	sachin	5000	data

SCHEMA :-
=========

=> a user in oracle db is called schema
=> objects created by user are called schema objects

SERVER
DATABASE
USER
TABLES
DATA

SERVER
FREE
SYS/TIGER (DBA)
SYSTEM/TIGER (DBA)

23-sep-26

How to connect to oracle :-
==========

=> open sqlplus and enter username and password

USERNAME :- SYSTEM
PASSWORD :- TIGER

OR

USERNAME :- SYSTEM/TIGER

creating user / schema / account :-
==========

=> only DBAs are having permissions to create new user

step 1 :- connect as dba

USERNAME :- SYSTEM/TIGER

step 2 :-

SQL>ALTER SESSION SET "_ORACLE_SCRIPT" = TRUE ;

step 3 :- create user

CREATE USER <name> IDENTIFIED BY <pwd>
DEFAULT TABLESPACE USERS

QUOTA UNLIMITED ON USERS ;

Ex :-

```
SQL>CREATE USER BATCH60  IDENTIFIED BY  ORACLE
        DEFAULT TABLESPACE USERS
        QUOTA UNLIMITED ON USERS ;
```

step 4 :- granting permissions to user

```
SQL>GRANT CONNECT,RESOURCE TO BATCH60 ;
```

connect => to connect to db
resource => to create tables

Changing password :-
==========

=> both user & dba can change password

by user :- (BATCH60/ORACLE)
========

```
SQL>PASSWORD
Changing password for BATCH60
Old password: ORACLE
New password: NARESH
Retype new password: NARESH
Password changed
```

by DBA :-
========

```
SQL>ALTER USER BATCH60 IDENTIFIED BY  TIGER ;
```

download & install :-
==========

download :-
========

https://www.oracle.com/database/free/get-started/#windows

Datatypes in ORACLE :-
==========

=> a datatype specifies

1 type of the data allowed
2 amount of memory allocated

DATATYPES

CHAR NUMERIC DATE BINARY

ASCII UNICODE number(p) date bfile
number(p,s) timestamp blob
char nchar
varchar2 nvarchar2
long nclob
clob

char(size) :-
=========

=> allows character data upto 2000 chars
=> recommended for fixed length char fields

ex :- NAME CHAR(10)

SACHIN - - - -
wasted

RAVI - - - - - -
wasted

=> In char datatype extra bytes are wasted , so char is not recommended for varible length fields and use char for fixed length fields

ex :- GENDER CHAR(1)

M
F

STATE_CODE CHAR(2)

AP
TG
MH

COUNTRY_CODE CHAR(3)

USA
IND

VARCHAR2(size) :-
==========

=> allows character data upto 4000 chars
=> recommended for variable length char fields

ex :- NAME VARCHAR2(10)

SACHIN - - - -
released

=> both char/varchar2 allows ascii chars (256 chars) includes a-z,A-Z,0-9,special chars
i.e. char/varchar2 allows alphanumeric data.

ex :- panno char(10)
password varchar2(12)
ifsc_code char(11)
emailid varchar2(20)

LONG :-
=======

=> allows character data upto 2GB

Ex :- TEXT LONG

CLOB :-
=======

=> character large object
=> allows character data upto 4GB

ex :- TEXT CLOB

NCHAR/NVARCHAR2/NCLOB :- (N => National)
==========

=> allows unicode chars (65536) that includes all ascii chars and also chars belongs to different languages.

=> ascii char occupies 1 byte and unicode char occupies 2 bytes

name	varchar2(20)	name	nvarchar2(20)
sachin	=> 6	sachin	=> 12

24-SEP-26

NUMBER(p) :-
==========

=> allows numbers without decimal (integer) up to 38 digits

ex :- EMPID NUMBER(4)

10
100
1000
10000 => not allowed

AADHARNO NUMBER(12)
PHONE NUMBER(10)
ACCNO NUMBER(11)

NUMBER(p,s) :-
==========

=> allows numbers with decimal (float)

p => precision => total no of digits allowed
s => scale => no of digits allowed after decimal

Ex :- SALARY NUMBER(7,2)

5000
5000.56
50000.55

500000.55 => NOT ALLOWED

5000.5678 => ALLOWED => 5000.57
5000.56378943 => ALLOWED => 5000.56

SAVG NUMBER(5,2)
BALANCE NUMBER

DATE :-
========

=> allows date & time
=> time is optional , if time not entered then oracle stores 12:00AM
=> default date format in oracle is DD-MON-YY / YYYY

EX :- DOB DATE

15-OCT-03 => 15-OCT-2003 => 15102003000000
5-FEB-98 => 05-FEB-2098
5-FEB-1998 => 05-FEB-1998

=> a date occupies 7 bytes

input :- DD-MON-YYYY

stores :- DD MM CC YY HH MI SS
1 1 1 1 1 1 1

TIMESTAMP :-
============

=> allows date,time and also milliseconds

EX :- T TIMESTAMP

24-SEP-2026 11:50:30.123
---------- --------- -----
DATE TIME MS

=> timestamp occupies 9 to 13 bytes depends on milliseconds

BINARY Types :-

==========

=> binary types allows multimedia objects like audio,video,images

1 BFILE (binary file)
2 BLOB (binary large object)

=> BFILE is called external lob because lob is stored outside db but db stores path
=> BLOB is called internal lob because lob is stored inside db .
=> if security is not important then use BFILE , if security is important then use BLOB

ex :- CPHOTO BFILE
MOVIE BLOB

==========

CREATING TABLES IN DB :-
==========

```
CREATE TABLE <tabname>
(
    colname  datatype(size) ,
    colname  datatype(size),
    ----------
);
```

Rules :-
========

1 name should start with alphabet
2 name should not contain spaces & special chars but allows _ $ #
3 name can be up to 128 chars
4 table can have max 1000 columns
5 no of rows unlimited

ex :- 123emp invalid
emp 123 invalid
emp*123 invalid
emp_123 valid

Ex :-

=> create a customer table with following fields ?

CUSTOMER
cid cname gender age city dor

```
CREATE TABLE customer
(
    cid         NUMBER(2),
    cname   VARCHAR2(10),
    gender   CHAR(1),
    age         NUMBER(2),
    city         VARCHAR2(10),
    dor         DATE
 );
```

above command is created table structure / definition / metadata that includes columns,datatype
and size.

DESC :- (DESCRIBE)
========

=> command used to see the structure of the table

syntax :- DESC <tabname>

Ex :-

SQL>DESC customer

Name	Type
CID	NUMBER(2)
CNAME	VARCHAR2(10)
GENDER	CHAR(1)
AGE	NUMBER(2)
CITY	VARCHAR2(10)
DOR	DATE

TABLE = STRUCTURE (columns) + DATA (rows)
(create) (insert)

INSERTING DATA INTO TABLE :-
==========

=> "INSERT" command is used to insert data into table
=> insert command creates a new row
=> using insert we can insert

1 single row
2 multiple rows

inserting single row :-
==========

INSERT INTO <tabname> VALUES(v1,v2,v3,---------);

Ex :-

SQL>INSERT INTO customer VALUES(10,'sachin','m',35,'Hyderabad','12-JAN-20');
SQL>INSERT INTO customer VALUES(11,'ananya','f',24,'Banglore',sysdate);

inserting multiple rows :-
==========

SQL>INSERT INTO customer
VALUES(12,'vijay','m',32,'Hyderabad','15-OCT-19') ,
(13,'priya','f',28,'Mumbai','20-AUG-22');

25-sep-26

inserting nulls :-
==========

=> a null means blank of empty
=> it is not equal to 0 or space
=> we insert nulls when value is absent or unknown
=> nulls can be inserted in two ways

method 1 :-
========

SQL>INSERT INTO customer VALUES(14,'suresh','m',NULL,'','08-MAY-18');

method 2 :-
=========

SQL>INSERT INTO customer(cid,cname,gender,dor) VALUES(15,'kavya','f',sysdate);

remaining two fields age,city are filled with nulls

=> above insert commands inserted data into instance (ram) which is temporary storage , to save this execute commit , after commit data will be copied to db.

SQL>COMMIT ;

customer table with sample data :-
==========

```
CREATE TABLE CUSTOMER
(
   cid     NUMBER(5),
   cname   VARCHAR2(50),
   gender  VARCHAR2(10),
   age     NUMBER(3),
   city    VARCHAR2(30),
   dor     DATE
);

INSERT INTO CUSTOMER VALUES (101, 'Aarav Sharma',     'MALE',   28, 'Hyderabad',  DATE
'2022-01-15');
INSERT INTO CUSTOMER VALUES (102, 'Ananya Reddy',      'FEMALE', 25, 'Bengaluru',
DATE '2022-03-20');
INSERT INTO CUSTOMER VALUES (103, 'Rahul Kumar',       'MALE',   35, 'Chennai',    DATE
'2021-07-12');
INSERT INTO CUSTOMER VALUES (104, 'Priya Nair',        'FEMALE', 31, 'Kochi',      DATE
'2023-02-18');
INSERT INTO CUSTOMER VALUES (105, 'Vijay Rao',         'MALE',   42, 'Hyderabad',  DATE
'2020-11-25');
INSERT INTO CUSTOMER VALUES (106, 'Sneha Reddy',       'FEMALE', 27, 'Vijayawada',
DATE '2022-06-10');
INSERT INTO CUSTOMER VALUES (107, 'Arjun Singh',       'MALE',   24, 'Delhi',      DATE
'2024-01-05');
INSERT INTO CUSTOMER VALUES (108, 'Kavya Sharma',      'FEMALE', 29, 'Mumbai',
DATE '2023-08-14');
INSERT INTO CUSTOMER VALUES (109, 'Suresh Babu',       'MALE',   45, 'Chennai',    DATE
```

```
'2019-05-22');
INSERT INTO CUSTOMER VALUES (110, 'Divya Menon',       'FEMALE', 33, 'Kochi',      DATE
'2021-09-30');

INSERT INTO CUSTOMER VALUES (111, 'Rohit Verma',       'MALE',   30, 'Pune',       DATE
'2022-12-11');
INSERT INTO CUSTOMER VALUES (112, 'Lakshmi Devi',      'FEMALE', 38, 'Hyderabad',
DATE '2020-03-17');
INSERT INTO CUSTOMER VALUES (113, 'Kiran Kumar',       'MALE',   26, 'Bengaluru',  DATE
'2024-04-08');
INSERT INTO CUSTOMER VALUES (114, 'Pooja Patel',       'FEMALE', 24, 'Ahmedabad',
DATE '2023-05-19');
INSERT INTO CUSTOMER VALUES (115, 'Manoj Reddy',       'MALE',   39, 'Warangal',   DATE
'2021-01-23');
INSERT INTO CUSTOMER VALUES (116, 'Swathi Rao',        'FEMALE', 28, 'Visakhapatnam',
DATE '2022-10-16');
INSERT INTO CUSTOMER VALUES (117, 'Aditya Gupta',      'MALE',   32, 'Delhi',      DATE
'2023-11-07');
INSERT INTO CUSTOMER VALUES (118, 'Neha Kapoor',       'FEMALE', 26, 'Mumbai',
DATE '2024-02-21');
INSERT INTO CUSTOMER VALUES (119, 'Prakash Naidu',     'MALE',   47, 'Visakhapatnam',
DATE '2019-08-13');
INSERT INTO CUSTOMER VALUES (120, 'Meena Kumari',      'FEMALE', 41, 'Jaipur',     DATE
'2020-06-29');

INSERT INTO CUSTOMER VALUES (121, 'Ravi Teja',         'MALE',   29, 'Hyderabad',  DATE
'2023-01-14');
INSERT INTO CUSTOMER VALUES (122, 'Sowmya Reddy',      'FEMALE', 30, 'Bengaluru',
DATE '2022-04-25');
INSERT INTO CUSTOMER VALUES (123, 'Naveen Kumar',      'MALE',   36, 'Chennai',    DATE
'2021-10-09');
INSERT INTO CUSTOMER VALUES (124, 'Harini Rao',        'FEMALE', 23, 'Vijayawada', DATE
'2024-03-15');
INSERT INTO CUSTOMER VALUES (125, 'Mahesh Babu',       'MALE',   44, 'Hyderabad',
DATE '2020-12-20');
INSERT INTO CUSTOMER VALUES (126, 'Aishwarya Nair',    'FEMALE', 32, 'Kochi',      DATE
'2023-07-06');
INSERT INTO CUSTOMER VALUES (127, 'Vamsi Krishna',     'MALE',   27, 'Guntur',     DATE
'2022-08-18');
INSERT INTO CUSTOMER VALUES (128, 'Deepika Singh',     'FEMALE', 35, 'Delhi',      DATE
'2021-03-27');
INSERT INTO CUSTOMER VALUES (129, 'Srikanth Reddy',    'MALE',   40, 'Warangal',   DATE
'2020-09-11');
```

```
INSERT INTO CUSTOMER VALUES (130, 'Nandini Sharma',     'FEMALE', 28, 'Pune',
DATE '2024-05-12');
COMMIT;
```

OPERATORS IN ORACLE :-
==========

Arithmetic Operators	=>	+ - * /
Relational Operators	=>	= > >= < <= = <> !=
Logical Operators	=>	AND OR NOT
Special Operators	=>	BETWEEN
		IN
		LIKE
		IS
		ANY
		ALL
		EXISTS
		PIVOT
Set Operators	=>	UNION
		UNION ALL
		INTERSECT
		MINUS
Concatenation Operator	=>	||

Displaying Data :-
==========

=> "SELECT" command is used to display data from table
=> we can display all rows and all columns
=> we can display specific rows and specific columns

```
SELECT  columns  /  *     FROM  tabname ;
```

```
SQL          =   ENGLISH
QUERIES      =   SENTENCES
CLAUSES      =   WORDS
```

```
SELECT  clause   => specify column names
FROM  clause     => specify table name
```

Ex :-

=> display customer names and city ?

SQL>SELECT cname,city FROM customer ;

=> display customer name,gender,age ?

SQL>SELECT cname,gender,age FROM customer ;

=> display all the data from customer table ?

SQL>SELECT * FROM customer ;

* => all columns

WHERE clause :-
==========

=> where clause is used to display specific row/rows from table

```
SELECT columns
FROM  tabname
WHERE condition ;
```

condition :-
=========

COLNAME OP VALUE

=> condition is a relational expression that returns TRUE / FALSE.
=> OP must be any relational operator like = > >= < <= <>
=> if cond = true row is selected , if false row is not selected

Ex :-

=> display customer details whose cid = 13 ?

SQL>SELECT * FROM customer WHERE cid = 13 ;

=> display customer details whose name is vijay ?

SQL>SELECT * FROM customer WHERE cname = 'vijay' ;

=> display customers more than 30 years old ?

SQL>SELECT * FROM customer WHERE age > 30 ;

=> customers registered after 2020 ?

SQL>SELECT * FROM customer WHERE dor > 2020 ;   => ERROR

SQL>SELECT * FROM customer WHERE dor > '31-DEC-2020' ;

=> customers registered before 2020 ?

SQL>SELECT * FROM customer WHERE dor < '01-JAN-2020' ;

=> customers not staying in hyderabad ?

SQL>SELECT * FROM customer WHERE city <> 'Hyderabad' ;

Compound Condition :-
==========

=> multiple conditions combined with AND / OR operators is called compound condition

WHERE cond1	AND	cond2	result
T		T	T
T		F	F
F		T	F
F		F	F

WHERE cond1	OR	cond2	result
T	T	T	
T	F	T	
F		T	T
F		F	F

=> list of customers whose id = 10,13,15 ?

SQL>SELECT * FROM customer WHERE cid = 10 OR cid = 13 OR cid=15 ;

=> customers staying in hyderabad,banglore ?

SQL>SELECT * FROM customer WHERE city='Hyderabad' OR city='Banglore' ;

=> display male customers more than 30 years old ?

```
SQL>SELECT * FROM customer WHERE gender='m'  AND  age>30 ;
```

28-sep-26

=> display list of customers age greather than 30 and less than 40 ?

```
 SQL>SELECT * FROM customer WHERE age >30 AND age < 40 ;
```

=> list of customers registered in 2026 ?

```
 SQL>SELECT *
        FROM customer
        WHERE  dor >= '01-JAN-26'  AND  dor <= '31-DEC-2026' ;
```

=> display female customers styaing in banglore ?

```
SQL>SELECT *
        FROM customer
        WHERE  gender='f' AND city='Banglore' ;
```

=> display male customers staying in Hyderabad age greather than 30 ?

```
SQL>SELECT *
       FROM customer
       WHERE gender='m'
                AND
               city = 'Hyderabad'
               AND
               age > 30 ;
```

=> list of male,female customers staying in hyderabad ?

```
 SQL>SELECT *
        FROM customer
        WHERE gender='f' OR gender='m'  AND  city='Hyderabad' ;
                ----------        ----------------------------------------------
```

above query returns male,female customers styaing hyderabad,banglore,mumbai becuase operator AND has got more priority than operator OR , To overcome this group conditions using ( ).

```
SQL>SELECT *
```

```
        FROM customer
        WHERE  (gender='f' OR gender='m')  AND  city='Hyderabad' ;
                  ----------------------------------          ------------------------
```

=> list of female customers staying in Hyderabad,Banglore ?

```
  SQL>SELECT *
        FROM customer
        WHERE  gender='f'  AND (city='Banglore' OR city='Hyderabad') ;
```

=>

STUDENT

sno	sname	s1	s2	s3
1	A	80	90	70
2	B	30	60	50
3	C	60	30	20
4	D	10	20	30

```
CREATE TABLE student
 (
   SNO number(2),
  sname varchar2(10),
   s1 number(3),
   s2  number(3),
   s3  number(3)
  );
```

```
SQL> INSERT INTO student VALUES(1,'A',80,90,70),(2,'B',30,60,50),
                                        (3,'C',60,30,20),(4,'D',10,20,30);
```

list of students who are passed ?

```
SQL>SELECT *
       FROM student
      WHERE s1>=35 AND s2>=35 AND s3>=35 ;
```

list of students who are failed ?

```
SQL>SELECT *
       FROM student
```

```
        WHERE s1<35 OR s2<35 OR s3<35 ;
```

=> list of students who are failed in exactly 1 subject ?

```
 SQL>SELECT *
        FROM student
        WHERE  (s1<35 AND s2>=35 AND s3>=35)
                    OR
                    (s1>=35 AND s2<35 AND s3>=35)
                    OR
                    (s1>=35 AND s2>=35 AND s3<35) ;
```

=> list of students who are failed in exactly 2 subjects ?

```
SQL>SELECT *
        FROM student
        WHERE  (s1<35 AND s2<35 AND s3>=35)
                    OR
                    (s1<35 AND s2>=35 AND s3<35)
                    OR
                    (s1>=35 AND s2<35 AND s3<35) ;
```

=> list of students who are failed in all 3 subjects ?

```
 SQL>SELECT *
        FROM student
        WHERE  s1<35 AND s2<35 AND s3<35 ;
```

IN operator :-

==========

=> use IN operator for list comparison

=> use IN operator for "=" comparison with multiple values

```
        WHERE  colname  =  v1,v2,v3,----          => INVALID
        WHERE  colname  IN  (v1,v2,v3,---)       =>  VALID
```

Ex :-

=> display customers whose id = 10,13,15 ?

```
 SQL>SELECT *
        FROM customer
```

```
        WHERE cid  IN (10,13,15) ;
```

=> list of customers staying Hyderbad,Banglore ?

```
 SQL>SELECT *
        FROM customer
        WHERE city IN  ('Hyderabad','Banglore');
```

=> customers not staying in Hyderabad,Banglore ?

```
 SQL>SELECT *
        FROM customer
        WHERE city NOT IN  ('Hyderabad','Banglore');
```

BETWEEN operator :-
==========

=> use between operator for range comparison

```
            list  => 10,20,30,40,50
            range => 10 to 50

     WHERE  colname  BETWEEN  v1  AND  v2     ( >= v1  AND   <= v2)
```

=> list of customers age between 20 and 30 ?

```
 SQL>SELECT *
        FROM customer
        WHERE age BETWEEN 20 AND 30 ;
```

=> list of customers registered in 2020 ?

```
 SQL> SELECT *
         FROM customer
        WHERE  dor  BETWEEN  '01-jan-2020' AND '31-dec-2020' ;
```

=> not registered in 2020 ?

```
SQL>  SELECT *
         FROM customer
        WHERE  dor  NOT BETWEEN  '01-jan-2020' AND '31-dec-2020' ;
```

=> display male customer

staying in hyderabad,banglore
age between 30 and 40
not registered in 2020 ?

```
SQL>SELECT *
        FROM customer
        WHERE  gender='m'
                    AND
                    city  IN ('Hyderabad','Banglore')
                    AND
                    age BETWEEN 30 AND 40
                    AND
                    dor NOT BETWEEN '01-JAN-2020' AND '31-DEC-2020' ;
```

=> list of samsung,redmi,realme mobile phones price between 10000 and 20000 ?

PRODUCTS
prodid pname price category brand

```
SQL>SELECT *
        FROM products
        WHERE  brand IN ('SAMSUNG','REDMI','REALME')
                    AND
                    price BETWEEN 10000 AND 20000
                    AND
                    category='MOBILE' ;
```

29-SEP-26

LIKE operator :-
==========

=> use LIKE operator for comparison with patterns

```
       WHERE  COLNAME   LIKE   'PATTERN'
```

=> pattern may consists of alphabets,digits,special chars and wildcard chars

wildcard chars :-
==========

% => 0 or many chars

_ => exactly 1 char

Ex :-

=> display customers name starts with 's' ?

SQL>SELECT * FROM customer WHERE cname LIKE 's%' ;

=> name ends with 'a' ?

SQL>SELECT * FROM customer WHERE cname LIKE '%a' ;

=> name contains 's' ?

SQL>SELECT * FROM customer WHERE cname LIKE '%s%' ;

=> where 'a' is the 4th char in their name ?

SQL>SELECT * FROM customer WHERE cname LIKE '___a%' ;

=> "a" is the 4th char from last ?

SQL>SELECT * FROM customer WHERE cname LIKE '%a___' ;

=> list of customers registered in october month ? dd-mon-yy

SQL>SELECT * FROM customer WHERE dor LIKE '%OCT%' ;

=> list of customers registered in 2020 ?

SQL>SELECT * FROM customer WHERE dor LIKE '%20' ;

IS operator :-
==========

=> use IS operator for NULL comparison

WHERE COLNAME IS NULL
WHERE COLNAME IS NOT NULL

Ex :-

=> list of customers whose age = null ?

SQL>SELECT * FROM customer WHERE age IS NULL ;

=> list of customers whose city is not = null ?

SQL>SELECT * FROM customer WHERE city IS NOT NULL ;

Ex :-

STUDENT

sno	sname	s1	s2	s3
1	A	80	90	70
2	B	null	60	50
3	C	60	null	null
4	D	null	null	null

=> list of students who are absent for any subject ?
=> list of students who are absent for all 3 subjects ?
=> list of students who are absent for 2 subjects ?
=> list of students who are absent for 1 subject ?
=> list of students who are present for all 3 subjects ?

```
CREATE TABLE EMP (
   EMPNO    NUMBER(4) PRIMARY KEY,
   ENAME    VARCHAR2(30),
   JOB      VARCHAR2(30),
   SAL      NUMBER(10,2),
   HIREDATE DATE,
   DEPT     VARCHAR2(20)
);

INSERT INTO EMP VALUES (1001, 'RAMESH', 'DEVELOPER', 35000, DATE '2020-01-15',
'IT');
INSERT INTO EMP VALUES (1002, 'SURESH', 'TESTER', 28000, DATE '2021-03-10', 'IT');
INSERT INTO EMP VALUES (1003, 'PRIYA', 'MANAGER', 65000, DATE '2018-06-20', 'HR');
INSERT INTO EMP VALUES (1004, 'LAKSHMI', 'ANALYST', 45000, DATE '2022-07-12',
'FINANCE');
INSERT INTO EMP VALUES (1005, 'VENKAT', 'DEVELOPER', 42000, DATE '2019-11-05', 'IT');
INSERT INTO EMP VALUES (1006, 'ANITA', 'CLERK', 22000, DATE '2023-02-14', 'HR');
INSERT INTO EMP VALUES (1007, 'RAHUL', 'TEAM LEAD', 75000, DATE '2017-09-18', 'IT');
INSERT INTO EMP VALUES (1008, 'KIRAN', 'TESTER', 30000, DATE '2022-04-25', 'IT');
INSERT INTO EMP VALUES (1009, 'SWATHI', 'ANALYST', 48000, DATE '2020-08-19',
'FINANCE');
```

```
INSERT INTO EMP VALUES (1010, 'ARJUN', 'MANAGER', 80000, DATE '2016-12-01',
'SALES');
INSERT INTO EMP VALUES (1011, 'DIVYA', 'DEVELOPER', 38000, DATE '2021-05-16', 'IT');
INSERT INTO EMP VALUES (1012, 'KAVYA', 'CLERK', 20000, DATE '2024-01-08', 'HR');
INSERT INTO EMP VALUES (1013, 'RAJESH', 'ANALYST', 52000, DATE '2019-04-11',
'FINANCE');
INSERT INTO EMP VALUES (1014, 'MANOJ', 'SALESMAN', 32000, DATE '2022-10-17',
'SALES');
INSERT INTO EMP VALUES (1015, 'POOJA', 'MANAGER', 70000, DATE '2018-02-28', 'HR');
INSERT INTO EMP VALUES (1016, 'NAVEEN', 'DEVELOPER', 55000, DATE '2017-07-09', 'IT');
INSERT INTO EMP VALUES (1017, 'SNEHA', 'TESTER', 33000, DATE '2023-06-21', 'IT');
INSERT INTO EMP VALUES (1018, 'GANESH', 'SALESMAN', 29000, DATE '2021-12-13',
'SALES');
INSERT INTO EMP VALUES (1019, 'MEENA', 'ANALYST', 46000, DATE '2020-03-04',
'FINANCE');
INSERT INTO EMP VALUES (1020, 'VIKRAM', 'TEAM LEAD', 85000, DATE '2015-08-22', 'IT');
INSERT INTO EMP VALUES (1021, 'SANDHYA', 'CLERK', 24000, DATE '2022-09-15', 'HR');
INSERT INTO EMP VALUES (1022, 'PRAVEEN', 'DEVELOPER', 60000, DATE '2016-05-30',
'IT');
INSERT INTO EMP VALUES (1023, 'GEETHA', 'SALESMAN', 35000, DATE '2023-11-20',
'SALES');
INSERT INTO EMP VALUES (1024, 'KARTHIK', 'ANALYST', 50000, DATE '2019-01-25',
'FINANCE');
INSERT INTO EMP VALUES (1025, 'MADHAVI', 'MANAGER', 90000, DATE '2014-04-07', 'HR');
INSERT INTO EMP VALUES (1026, 'SRINIVAS', 'TESTER', 27000, DATE '2024-03-18', 'IT');
INSERT INTO EMP VALUES (1027, 'BHAVANA', 'DEVELOPER', 44000, DATE '2020-10-06',
'IT');
INSERT INTO EMP VALUES (1028, 'RAJ', 'SALESMAN', 31000, DATE '2022-06-14', 'SALES');
INSERT INTO EMP VALUES (1029, 'HARISH', 'ANALYST', 58000, DATE '2017-02-19',
'FINANCE');
INSERT INTO EMP VALUES (1030, 'ANJALI', 'CLERK', 21000, DATE '2025-01-10', 'HR');
commit;
```

1.

Display all employees working in the IT department.

2.

Display employees whose salary is greater than 40000.

3.

Display employees whose salary is less than 30000.

4.

Display employees whose salary is equal to 50000.

5.

Display employees whose salary is greater than or equal to 60000.

6.

Display employees whose salary is less than or equal to 25000.

7.

Display employees who do not belong to the HR department.

8.

Display employees whose job is DEVELOPER.

9.

Display employees whose department is FINANCE.

10.

Display employees whose employee number is greater than 1015.

11.

Display employees whose salary is between 30000 and 50000.

12.

Display employees whose salary is between 40000 and 70000.

13.

Display employees whose employee number is between 1005 and 1015.

14.

Display employees hired between 1 January 2020 and 31 December 2022.

15.

Display employees whose salary is not between 20000 and 35000.

16.

Display employees hired between 1 January 2023 and 31 December 2025.

17.

Display IT employees whose salary is between 35000 and 60000.

18.

Display employees in the HR department whose salary is between 20000 and 50000.

19.

Display employees working in IT, HR or SALES.

20.

Display employees whose job is either DEVELOPER or TESTER.

21.

Display employees whose department is either FINANCE or HR.

22.

Display employees whose salary is 30000, 45000 or 60000.

23.

Display employees who do not work in IT or HR.

24.

Display employees whose job is MANAGER, ANALYST or TEAM LEAD.

25.

Display employees working in SALES or FINANCE with a salary greater than 30000.

26.

Display employees whose department is not IT, FINANCE or SALES.

27.

Display employees whose names start with the letter R.

28.

Display employees whose names end with the letter A.

29.

Display employees whose names contain the letter V.

30.

Display employees whose names start with S and end with A.

31.

Display employees whose names have exactly five characters.

32.

Display employees whose names have R as the second character.

33.

Display employees whose names start with K and contain A.

34.

Display employees whose names do not start with A.

35.

Display employees whose job titles contain the letter E.

36.

Display employees whose names contain the substring RA.

37.

Display employees whose JOB is NULL.

38.

Display employees whose DEPT is NULL.

39.

Display employees whose SAL is NULL.

40.

Display employees whose JOB is not NULL.

41.

Display employees whose DEPT is not NULL.

42.

Display employees whose SAL is not NULL.

43.

Display employees whose DEPT is NULL or whose JOB is NULL.

44.

Display employees whose salary is not NULL and is greater than 40000.
Level 6 — AND, OR and NOT combinations

45.

Display IT employees whose salary is greater than 40000.

46.

Display employees working in HR whose salary is less than 30000.

47.

Display employees who work in IT and have the job DEVELOPER.

48.

Display employees who work in HR or SALES.

49.

Display employees whose salary is greater than 50000 and who were hired after 1 January 2018.

50.

Display employees who work in IT or FINANCE and earn less than 50000.

51.

Display employees whose job is DEVELOPER or TESTER and whose salary is greater than 30000.

52.

Display employees who do not work in HR and earn more than 40000.

53.

Display employees hired between 2020 and 2023 who work in IT or SALES.

54.

Display employees whose names start with R and whose salary is between 30000 and 60000.
Level 7 — Advanced WHERE clause challenges

These combine multiple operators in a single query.

55.

Display employees from IT, HR and FINANCE whose salary is between 30000 and 70000.

56.

Display employees whose names start with R or S and whose salary is greater than 30000.

57.

Display employees hired after 1 January 2020, excluding those working in HR.

58.

Display employees whose job is DEVELOPER, TESTER or ANALYST and whose salary is between 30000 and 60000.

59.

Display employees whose department is IT or FINANCE, whose salary is greater than 40000, and whose name contains A.

60.

Display employees whose name starts with A, K or R and whose hire date is between 2018 and 2024.

61.

Display employees who are not managers and whose salary is not between 30000 and 50000.

62.

Display employees whose department is not NULL and whose job is either ANALYST or DEVELOPER.

63.

Display employees whose salary is greater than 45000 and who were hired before 1 January 2020, excluding FINANCE employees.

64.

Display employees whose name has exactly five characters, whose salary is greater than 25000, and whose department is IT or SALES.

summary :-
========

```
WHERE  COLNAME  IN (V1,V2,V3,---)
WHERE  COLNAME  BETWEEN V1 AND V2
WHERE  COLNAME LIKE  'PATTERN'
WHERE  COLNAME IS NULL
```

ALIAS :-
======

=> alias means another name or alternative name
=> used to change column heading

COLNAME / EXPR   [AS]   ALIAS

Ex :-

=> display ENAME   ANNUAL SALARY ?

```
SQL>SELECT ename,sal*12  as annsal FROM emp ;

SQL>SELECT ename,sal*12  as "ANNUAL SAL" FROM emp ;
```

=> display ENAME HIREDATE EXPERIENCE in years ?

experience = (SYSDATE - hiredate)/365

```
SQL>SELECT ename,hiredate, (SYSDATE-hiredate)/365 as experience FROM emp ;
```

=> display only the employees having more than 5 years of experience ?

```
SQL>SELECT ename,hiredate, (SYSDATE-hiredate)/365 as experience
      FROM emp
      WHERE  (SYSDATE-hiredate)/365   > 5 ;
```

=> display ENAME SAL HRA DA TAX TOTSAL ?

HRA = house rent allowance = 20% on sal
DA = dearness allowance = 40% on sal
TAX = 10% on sal

TOTSAL = SAL + HRA + DA - TAX

```
SQL>SELECT ename,sal,
                sal*0.2 as hra,
                sal*0.4 as da,
                sal*0.1 as tax,
                sal+(sal*0.2) + (sal*0.4) - (sal*0.1) as totsal
        FROM emp;
```

RAMESH 35000 7000 14000 3500 52500

ORDER BY clause :-
==========

=> order by clause is used to sort table data
=> we can sort data based on one or more columns
=> we can sort either in ascending or in descending order

	ASC	DESC
NUMBER	0-9	9-0
CHAR	A-Z	Z-A
DATE	OLD-NEW	NEW-OLD

syntax :-
=======

```
SELECT columns
FROM tabname
[WHERE cond]
ORDER BY  col1 ASC / DESC , col2 ASC/DESC,-----
```

=> default order is ASC

Ex :-

=> arrange employee list name wise ascending ?

```
SQL>SELECT  ename,sal,hiredate
        FROM emp
        ORDER BY ename ASC ;
```

=> arrange employee list sal wise desc order ?

```
SQL>SELECT *
        FROM emp
        ORDER BY sal  DESC ;
```

```
 1    A       3000                  4       D       8000
 2    B       6000                  2       B       6000
 3    C       4000 ================>  5     E       5000
 4    D       8000                   3      C       4000
 5    E       5000                   1      A       3000
```

=> arrange employee list and employee who joined first display first ?

```
SQL>SELECT ename,sal,hiredate
        FROM emp
        ORDER BY  hiredate  ASC ;
```

30-sep-26

sorting based on multiple columns :-
==========

=> arrange employee list dept wise asc and with in dept sal wise desc ?

```
SQL>SELECT ename,sal,dept
       FROM emp
       ORDER BY dept ASC , sal DESC ;
```

```
        A    4000   IT                 F       9000   HR
        B   3000   SALES                      C       6000   HR
        C   6000   HR                 E       6000   IT
        D   8000   SALES  ============>       A       4000   IT
        E   6000   IT                         D       8000   SALES
        F   9000   HR                 B       3000   SALES
```

=> arrange employee list dept wise and with in dept hiredate wise asc ?

```
SQL> SELECT ename,hiredate,dept
        FROM emp
        ORDER BY dept ASC,hiredate ASC ;
```

=> arrange student list avg wise desc
if avg same then arrange m wise desc
if m also same then arrange p wise desc ?

STUDENT

sno	sname	m	p	c
1	A	80	90	70
2	B	60	70	50
3	C	90	70	80
4	D	90	80	70

```
SQL> SELECT  sno,sname,m,p,c, (m+p+c)/3  as avg
        FROM student
        ORDER BY   (m+p+c)/3  DESC ,m  DESC,p  DESC ;
```

4	D	90	80	70
3	C	90	70	80
1	A	80	90	70
2	B	60	70	50

=> display employees working for HR,IT dept and arrange output sal wise desc ?

```
SQL>SELECT ename,sal,dept
        FROM emp
        WHERE dept IN ('HR','IT')
        ORDER BY sal DESC ;
```

Execution :-

```
 FROM
WHERE
SELECT
ORDER BY
```

FROM emp :-
==========

1	A	4000	HR
2	B	6000	IT
3	C	5000	SALES
4	D	7000	HR
5	E	9000	IT

WHERE dept IN ('HR','IT') :-
==========

1	A	4000	HR
2	B	6000	IT
4	D	7000	HR
5	E	9000	IT

SELECT ename,sal,dept :-
==========

A	4000	HR
B	6000	IT
D	7000	HR
E	9000	IT

ORDER BY sal DESC :-
==========

E	9000	IT
D	7000	HR
B	6000	IT
A	4000	HR

=> sept month statement of particular customer
display latest transaction first ?

TRANSACTIONS

trid	ttype	tdate	tamt	accno
1	w	30-	2000	100
2	d	30-	5000	100

```
SQL>SELECT *
      FROM transactions
      WHERE  accno = 100
                  AND
                  tdate  BETWEEN '01-SEP-2026'  AND  '30-SEP-2026'
      ORDER BY  tdate DESC ;
```

==========
======

DISTINCT clause :-
==========

=> distinct clause eliminates duplicates from select query output

```
SELECT  DISTINCT  col1
SELECT  DISTINCT  col1,col2,---
SELECT  DISTINCT  *
```

Ex :-

```
SQL>SELECT DISTINCT  DEPT  FROM emp ;

        IT
        HR
        FINANCE
        SALES

SQL>SELECT DISTINCT JOB FROM EMP ;

        DEVELOPER
        TESTER
        MANAGER
        ANALYST
        CLERK
        TEAM LEAD
        SALESMAN
```

FETCH clause :-
==========

=> fetch clause is used to display top n rows

```
SELECT columns
FROM tabname
[WHERE cond]
[ORDER BY  ---]
[OFFSET  <n> ROWS]  FETCH  FIRST / NEXT  <n>  ROW/ROWS  ONLY  ;
```

Ex :-

=> display first 5 rows from emp table ?

```
SQL>SELECT empno,ename,sal
        FROM emp
        FETCH FIRST 5  ROWS ONLY ;
```

=> display 5th rows only ?

```
SQL>SELECT empno,ename,sal
        FROM emp
        OFFSET 4 ROWS FETCH  NEXT 1 ROW ONLY ;
```

=> display 5th row to 10th row ?

```
SQL>SELECT empno,ename,sal
        FROM emp
        OFFSET 4 ROWS FETCH  NEXT 6 ROWS ONLY ;
```

=> display TOP 3 highest paid employees ?

```
 SQL> SELECT empno,ename,sal
         FROM emp
         ORDER BY sal DESC
         FETCH FIRST 3 ROWS ONLY ;
```

=> display top 3 max salaries ?

```
SQL> SELECT  sal
         FROM emp
         ORDER BY sal DESC
         FETCH FIRST 3 ROWS ONLY ;
```

=> display 5th max salary ?

```
SQL>SELECT  sal
         FROM emp
         ORDER BY sal DESC
         OFFSET 4 ROWS FETCH NEXT 1 ROW ONLY ;
```

=> display top 3 employees based on experience ?

```
SQL>SELECT empno,ename,hiredate
        FROM emp
       ORDER BY hiredate ASC
        FETCH FIRST 3 ROWS ONLY ;
```

=> display top 3 experienced employees in IT dept ?

```
SQL>SELECT empno,ename,hiredate,dept
        FROM emp
        WHERE dept='IT'
        ORDER BY hiredate ASC
        FETCH FIRST 3 ROWS ONLY ;
```

summary :-

```
WHERE                =>  To select specific rows
ORDER BY   => To sort table data
DISTINCT     =>  To eliminate duplicates
FETCH                =>  To display top n rows
```

USER_TABLES :-
==========

=> It is a system table that stores information about tables created by user

display list of tables created by user ?

```
SQL>SELECT TABLE_NAME FROM USER_TABLES ;

SQL>SELECT TABLE_NAME FROM USER_TABLES WHERE TABLE_NAME LIKE  'E%' ;
```

ALL_USERS :-
==========

=> It is a system table that stores information about users

display list of users in oracle db ?

```
SQL>SELECT USERNAME FROM ALL_USERS ;
```

==========

DML commands :- (Data Manipulation Lang)
==========

INSERT
UPDATE
DELETE
MERGE
INSERT ALL

TABLE = STRUCTURE (COLUMNS) + DATA (ROWS)

DDL DML

=> DML commands acts on table data
=> DML commands acts on instance i.e. temporary storage allocated in ram
=> to save the operation execute COMMIT.
=> to cance the operation execute ROLLBACK.

UPDATE command :-
==========

=> command used to modify table data
=> we can update all rows or specific rows
=> we can update single column or multiple columns

```
UPDATE  tabname
SET  colname =  value ,  colname = value , ----------
[WHERE cond] ;
```

Ex :-

=> update all emplyoees sal with 50000 ?

```
SQL>UPDATE emp SET sal = 50000 ;
```

=> update employee sal with 40000 whose empno = 1001 ?

```
SQL>UPDATE emp SET sal = 40000 WHERE empno = 1001 ;
```

=> update sal with 50000 and job with ANALYST whose empno = 1001 ;

```
SQL>UPDATE emp SET sal=50000,job='ANALYST'  WHERE empno = 1001;
```

=> increment salaries of the developer by 10% ?

SQL>UPDATE emp SET sal = sal + (sal*0.1) WHERE job='DEVELOPER' ;

=> transfer employees from FINANCE dept to SALES dept ?

SQL>UPDATE emp SET dept = 'SALES' WHERE dept = 'FINANCE' ;

=> increase the price of samsung,redmi,realme mobiles by 10% ?

PRODUCTS
prodid pname price category brand

```
SQL>UPDATE products
        SET price = price*1.1
         WHERE category='mobiles'
                AND
                brand IN ('samsung','redmi','realme') ;
```

DELETE command :-
==========

=> command used to delete row/rows
=> we can delete all rows or specific rows

```
    DELETE FROM <tabname> [WHERE cond] ;
```

Ex :-

=> delete all rows from emp table ?

SQL>DELETE FROM emp ;

=> deletes employees working as DEVELOPER,TESTER ?

SQL>DELETE FROM emp WHERE job IN ('DEVELOPER','TESTER') ;

=> delete the employees having more than 10 years of expr ?

SQL>DELETE FROM emp WHERE (SYSDATE - hiredate)/365 > 10 ;

=> delete employees joined between 2020 and 2022 ?

SQL>DELETE FROM emp WHERE hiredate BETWEEN '01-JAN-2020' AND '31-DEC-2022' ;

=> delete employees having more than 10LAC annual salary ?

SQL>DELETE FROM emp WHERE sal*12 > 1000000

FLASHBACK :-
==========

 => using FLASHBACK we can access the data that exists some time back
 => a normal query returns current data
 => a query that returns past data is called flashback query
 => useful to recover data after commit.

  SELECT columns
  FROM tabname
  AS OF TIMESTAMP (SYSDATE - INTERVAL)

 Ex :-

 => the following query displays 5 mins back data in emp table ?

  SELECT *
  FROM emp
  AS OF TIMESTAMP (SYSDATE - INTERVAL '5' MINUTE) ;

How to recover data after commit :-
==========

step 1 :-

 SQL>DELETE FROM emp ;

step 2 :-

 SQL> COMMIT ;

step 3 :-

SQL>INSERT INTO emp
  SELECT *
  FROM emp
  AS OF TIMESTAMP (SYSDATE - INTERVAL '5' MINUTE) ;

2-oct-26

DDL commands :- (Data Definition Lang)
==========

CREATE
ALTER
DROP
TRUNCATE
RENAME
FLASHBACK
PURGE

=> DDL commands acts on table structure (columns,datatype and size)
=> all DDL commands are auto committed.

DDL command = DDL command + COMMIT

ex 1 :-

```
CREATE TABLE a(a  NUMBER(2));
INSERT INTO a VALUES(10);
INSERT INTO a VALUES(20);
INSERT INTO a VALUES(30);
ROLLBACK ;
```

create table a => saved
insert 10,20,30 => cancelled

ex 2 :-

```
CREATE TABLE a(a  NUMBER(2));   => commit
INSERT INTO a VALUES(10);
INSERT INTO a VALUES(20);
CREATE TABLE b(b NUMBER(2));   => commit
INSERT INTO a VALUES(30);
INSERT INTO a VALUES(40);
ROLLBACK ;
```

create table a ,b => saved
insert 10,20 => saved
insert 30,40 => cancelled

ALTER command :-

==========

=> command used to modify table structure
=> modifying structure means

1 adding columns
2 droping columns
3 rename column
4 modify column
changing datatype
changing size

Adding columns :-
==========

ALTER TABLE <tabname>
ADD (colname datatype(size) , colname datatype(size),----) ;

Ex :-

=> add column gender to emp table ?

SQL>ALTER TABLE emp
ADD (gender CHAR(1));

after adding by default the new column is filled with nulls , to insert data into the new column use update command.

SQL>UPDATE emp SET gender = 'M' WHERE empno = 1001 ;

SQL>UPDATE emp SET gender = '&gender' WHERE empno = &empno ;

Enter value for gender: M
Enter value for empno: 1002

1 row updated.

SQL> / => executes previous command
Enter value for gender: F
Enter value for empno: 1003

1 row updated

Droping columns :-
==========

ALTER TABLE <tabname>
DROP (col1,col2,----) ;

Ex :-

=> drop column gender from emp ?

SQL>ALTER TABLE emp
DROP (gender) ;

Renaming a column :-
==========

ALTER TABLE <tabname>
RENAME COLUMN <oldname> TO <newname> ;

Ex :-

=> rename column hiredate to doj ?

SQL>ALTER TABLE emp
RENAME COLUMN hiredate TO doj ;

SELECT empno,ename,hiredate as doj FROM emp ;

difference between alias and rename ?

=> alias changes column heading in select stmt output
=> rename changed column name in table

Modifying a column :-
==========

ALTER TABLE <tabname>
MODIFY (colname datatype(size));

1 changing datatype
2 changing size

=> increase the size of ename to 40 ?

```
ALTER TABLE emp
        MODIFY  (ename  VARCHAR2(40));
```

```
ALTER TABLE emp
        MODIFY  (ename  VARCHAR2(6));  => ERROR => some names contains more than 6
```
chars

NOTE :-

1 numeric field must be empty to decrease precision or scale

```
ALTER TABLE emp
        MODIFY (sal  NUMBER(9,2));   => ERROR
```

2 column must be empty to change datatype

```
ALTER TABLE emp
        MODIFY  (empno  VARCHAR2(10));   => ERROR
```

How to change datatype of a column if column is not empty :-
==========

modify column empno datatype to varchar2 ?

step 1 :- add new column empid VARCHAR2(10)

```
ALTER TABLE emp
        ADD (empid  VARCHAR2(10));
```

step 2 :- copy the values from empno to empid

```
UPDATE emp SET empid =  'TCS-'||empno  ;
```

step 3 :- remove column empno

```
ALTER TABLE emp
        DROP (empno) ;
```

step 4 :- rename column empid to empno

```
   ALTER TABLE emp
        RENAME  COLUMN empid TO empno  ;
```

DROP command :-
==========

=> command used to drop table from db.
=> drops table structure with data

```
    DROP TABLE <tabname>
```

Ex :-

```
SQL>DROP TABLE emp ;
```

after dropping the table is moved to recyclebin

=> To see the recyclebin

```
SQL>SHOW RECYCLEBIN ;
```

FLASHBACK command :-
==========

=> command used to restore the table from recyclebin

```
      FLASHBACK TABLE <tabname> TO BEFORE DROP;
```

Ex :-

```
  SQL>FLASHBACK TABLE emp TO BEFORE DROP ;
```

=> table is restored with rows & cols that exists before drop

PURGE command :-
==========

=> command used to delete the table from recyclebin
=> after deleting table from recyclebin we cannot flashback the table

```
       PURGE TABLE <tabname> ;
```

Ex :-

SQL>PURGE TABLE EMP ;

DROP & PURGE :-
==========

SQL>DROP TABLE CUSTOMER PURGE ;

=> above customer table is dropped and also deleted from recyclebin

Empty Recyclebin :-
==========

SQL>PURGE RECYCLEBIN ;

=> deletes all tables from recyclebin
=> recyclebin becomes empty

TRUNCATE command :-
==========

=> deletes all the data from table but keeps structure
=> will empty the table
=> releases memory allocated for table

TRUNCATE TABLE <tabname> ;

Ex :-

SQL>TRUNCATE TABLE emp ;

=> oracle goes to memory and releases all blocks allocated for table , when memory is released
then data stored in memory also deleted.

DROP VS DELETE VS TRUNCATE :-
==========

DROP	DELETE/TRUNCATE
drops table structure and data	deletes only data but not structure

	DELETE	TRUNCATE
1	DML	DDL
2	deletes all rows and specific rows	can delete only all rows but cannot delete specific rows
3	where cond can be used with delete	where cannot be used with truncate
4	can be rolledback	cannot be rolledback
5	deletes row-by-row	deletes all rows at a time
6	slower	faster
7	will not release memory	releases memory

RENAME :-
=========

=> command used to change table name .

RENAME <oldname> TO <newname> ;

Ex :-

SQL>RENAME emp TO employees ;

Built-in Functions in ORACLE :-
==========

=> a functions accepts some input performs some calculation and must return a value
=> functions are used

1 Data Cleaning
2 Data Transformation
3 Data Analysis
4 Calculations

Types of functions :-
==========

1 Single row functions
- character
- numeric
- date
- conversion
- special
- anallytical

2 Multi row functions

character functions :-
==========

UPPER() & LOWER() :-
==========

UPPER(string) => converts string to uppercase
LOWER(string) => converts string to lowercase

Ex :-

SQL>SELECT UPPER('hello') ; => HELLO

before 23ai :-
==========

SQL>SELECT UPPER('hello') FROM DUAL ; => HELLO

SQL>SELECT LOWER('HELLO'); => hello

=> display EMPNO ENAME SAL ? display names in lowercase ?

SQL>SELECT empno , LOWER(ename) as ename,sal FROM emp ;

=> convert names to lowercase in table ?

SQL>UPDATE emp SET ename = LOWER(ename) ;

INITCAP() :-
==========

=> converts initials into capitals

INITCAP(string)

Ex :-

SQL>SELECT INITCAP('hello welcome') ; => Hello Welcome

5-oct-26

LENGTH() :-
==========

=> returns string length i.e. no of chars

LENGTH(string)

EX :-

1 SQL>SELECT LENGTH('HELLO WELCOME'); => 13

2 SQL>SELECT ename,LENGTH(ename) as len FROM emp ;

smith 5
allen 5
ward 4

=> display employee list name contains 5 chars ?

SQL>SELECT * FROM emp WHERE ename LIKE '_____' ;

SQL>SELECT * FROM emp WHERE LENGTH(ename) = 5 ;

SQL>SELECT * FROM emp WHERE LENGTH(ename) < 5 ;

SUBSTR() :-
==========

=> returns part of the string

SUBSTR(string , start , [no of chars])

Ex :-

SQL>SELECT SUBSTR('HELLO WELCOME',1,5) ; => HELLO
SQL>SELECT SUBSTR('HELLO WELCOME',10,4); => COME

```
SQL>SELECT  SUBSTR('HELLO WELCOME',7);                  =>      WELCOME

SQL>SELECT  SUBSTR('HELLO WELCOME',-7,4);       =>      WELC
SQL>SELECT  SUBSTR('HELLO WELCOME',-4);         =>      COME
```

=> employees name starting with vowel ?

```
SQL>SELECT * FROM emp WHERE SUBSTR(ename,1,1)  IN  ('a','e','i','o','u');
```

=> name starts with consonant ?

```
SQL>SELECT * FROM emp WHERE SUBSTR(ename,1,1)  NOT IN  ('a','e','i','o','u');
```

=> name starts and ends with same char ?

```
SQL>SELECT * FROM emp WHERE SUBSTR(ename,1,1) =   SUBSTR(ename,-1,1) ;
```

=> generate emailids for employees ?

```
empno                ename  emailid
7369          smith    smi736@tcs.com
7499          allen    all749@tcs.com

SQL>SELECT empno ,ename,
                  SUBSTR(ename,1,3)||SUBSTR(empno,1,3)||'@tcs.com'  as emailid
        FROM emp ;
```

=> store emailids in db ?

step 1 :- add emailid column to emp table

```
SQL>ALTER TABLE emp
                ADD (emailid  VARCHAR2(20));
```

step 2 :- update the column with emailids

```
SQL>UPDATE emp SET emailid = SUBSTR(ename,1,3)||SUBSTR(empno,1,3)||'@tcs.com' ;
```

INSTR() :-

=========

=> returns character position in a string

INSTR(string,char,[start,occurance])

Ex :-

```
SQL>SELECT  INSTR('HELLO WELCOME','O') ;  =>   5
SQL>SELECT  INSTR('HELLO WELCOME','K') ;  =>   0

SQL>SELECT  INSTR('HELLO WELCOME','O',1,2);  =>  11
SQL>SELECT  INSTR('HELLO WELCOME','E',7,2);   =>  13

SQL>SELECT  INSTR('HELLO WELCOME','L',-7,2);   =>   3
SQL>SELECT  INSTR('HELLO WELCOME','E',-1,3);  =>   2
```

Ex 1 :-

CUST

cid	cname
10	sachin tendulkar
11	virat kohli

display CID FNAME LNAME ?

SUBSTR(string,start,[no of chars])
INSTR(string,char,[start,occurance])

FNAME = SUSBSTR(cname , 1 , INSTR(cname,' ')-1)

LNAME = SUBSTR(cname,INSTR(cname,' ')+1)

```
SQL>SELECT  cid ,
                  SUBSTR(cname , 1 , INSTR(cname,' ')-1) as fname,
                  SUBSTR(cname,INSTR(cname,' ')+1)    as lname
        FROM cust;
```

Ex 2 :- display CID FNAME MNAME LNAME ?

CUST

cid	cname
10	sachin ramesh tendulkar
11	virat anushka kohli

RPAD & LPAD :-
==========

=> both functions used to fill string with a char

RPAD(string,length,char) => fills on right side
LPAD(string,length,char) => fills on left side

Ex :-

```
SQL>SELECT RPAD('HELLO',10,'*');        =>  HELLO*****
SQL>SELECT LPAD('HELLO',10,'*');        =>  *****HELLO
SQL>SELECT RPAD('*',10,'*');            =>  **********
```

=> display ENAME SAL ?

```
XXX
XXXX
XXXXX
```

SQL>SELECT ename, RPAD('X',LENGTH(sal),'X') as sal FROM emp ;

```
smith   XXX
allen   XXXX
```

=>

accounts

accno	bal
123456789012	10000

output :-

your a/c no XXXX9012 debited ---- ?

method 1 :- RPAD('X',4,'X')||SUBSTR(accno,-4,4)

method 2 :- LPAD(SUBSTR(accno,-4,4) , 8 , 'X')

6-OCT-26

LTRIM,RTRIM,TRIM :-
==========

=> functions used to remove spaces and unwanted chars

LTRIM(str1 , [str2]) => removes on left side
RTRIM(str1,[str2]) => removes on right side
TRIM(string) => removes both sides

Ex :-

```
SELECT LTRIM('    HELLO    ')  =>    'HELLO    '
SELECT RTRIM('  HELLO    ')        =>       '    HELLO'
SELECT TRIM('    HELLO  ')    =>    'HELLO'

SELECT  LTRIM('@@@HELLO@@@', '@')              =>    HELLO@@@
SELECT  RTRIM('@@@HELLO@@@', '@')              =>    @@@HELLO
SELECT  TRIM(BOTH '@' FROM '@@@HELLO@@@')    => HELLO
```

REPLACE() :-
==========

=> function used to replace one string with another string

REPLACE(str1,str2,str3)

Ex:-

```
SQL>SELECT REPLACE('HELLO','ELL','ABC');    =>    HABCO
SQL>SELECT REPLACE('HELLO','L','ABC');       =>    HEABCABCO
SQL>SELECT REPLACE('HELLO','ELO','ABC');   =>    HELLO

SQL> SELECT  REPLACE('@@HE@@LL@@O@@','@','');     =>  HELLO
```

=> replace month MAY with APR in hiredate ?

```
SQL>UPDATE emp SET hiredate = REPLACE(hiredate,'MAY','APR') ;
```

TRANSLATE() :-
==============

=> used to translate one char to another char

TRANSLATE(str1,str2,str3)

Ex :-

```
SQL>SELECT TRANSLATE('HELLO','ELO','ABC');          =>    HABBC
```

E => A
L => B
O => C

SQL>SELECT TRANSLATE('HELLO','ELO',''); => NULL

=> translate function is used to encrypt data i.e. converting plain text to cipher text

display ENAME SAL ? encrypt salaries ?

SQL>SELECT ename , TRANSLATE(sal , '0123456789', '$bT*n@!%^#') as sal FROM emp;

jones 2975 T#%@

=> remove all special chars from '@!HE#$LL%^O*@' ?

SQL>SELECT
REPLACE(TRANSLATE('@!HE#$LL%^O*@' ,'@!#$%^*','@@@@@@@') ,'@','');

@@HE@@LL@@O@@

UPPER
LOWER
INITCAP
LENGTH
SUBSTR
INSTR
LPAD,RPAD,
LTRIM,RTRIM,TRIM
REPLACE
TRANSLATE

==========

Numeric Functions :-
==========

ABS() :-
======

=> returns absolute value

ABS(number)

SQL>SELECT ABS(-10) ; => 10

POWER() :-
==========

=> used to calculate power

SQL>SELECT POWER(3,2) ; => 9

SQRT() :-
========

=> returns square root of a number

SQL>SELECT SQRT(16) ; => 4

MOD() :-
=======

=> returns remainder

SQL>SELECT MOD(10,2) ; => 0

=> display employees earning multiples of 100 ?

SQL>SELECT * FROM emp WHERE MOD(sal,100) = 0 ;

Rounding numbers :-
================

ROUND
TRUNC
CEIL
FLOOR

ROUND
========

=> used to round number to integer or to decimal places
=> round function acts according to average

ROUND(number, [decimal places])

Ex :-

SQL>SELECT ROUND(38.4567) => 38

38----------38.5----------39

number >= avg => rounded to highest
number < avg => rounded to lowest

SQL>SELECT ROUND(38.5567); => 39

SQL>SELECT ROUND(38.4567,2); => 38.46

SQL>SELECT ROUND(38.4537,2); => 38.45

SQL>SELECT ROUND(38.4567,1); => 38.5

SQL>SELECT ROUND(38.4562,3); => 38.456

=> display ENAME EXPERIENCE in years ? round experience to integer ?

```
SQL> SELECT  ename,
             ROUND((SYSDATE - hiredate)/365)   as experience
     FROM emp ;
```

TRUNC() :-
=========

=> rounds number always to lowest

TRUNC(number,[decimal places])

Ex :-

SQL>SELECT TRUNC(38.5678); => 38

38----------39

SQL>SELECT TRUNC(38.5678,2); => 38.56

SQL>SELECT ROUND(386,-1); => 390

380----------385----------390

SQL> SELECT ROUND(386,-2); => 400

300----------350----------400

SQL>SELECT ROUND(386,-3); => 0

0 ----------500----------1000

SQL>SELECT ROUND(4567,-1) , ROUND(4567,-2) , ROUND(4567,-3) ;

O/P :- 4570 4600 5000

SQL>SELECT TRUNC(4567,-1) , TRUNC(4567,-2) , TRUNC(4567,-3) ;

O/P :- 4560 4500 4000

=> round employee salaries to hundred in emp table ?

SQL>UPDATE emp SET sal = ROUND(sal , -2) ;

CEIL() :-
========

=> rounds number always to highest

CEIL(number)

Ex :-

SQL>SELECT CEIL(3.1); => 4

FLOOR() :-
=========

=> rounds number always to lowest

FLOOR(number)

Ex :-

SQL>SELECT FLOOR(3.9); => 3

7-oct-26

DATE functions :-
==========

sysdate + 10 => 10 days added to sysdate
sysdate - 10 => 10 days subtracted from sysdate
sysdate - hiredate => returns difference between two dates in days
sysdate + hiredate => ERROR

EXTRACT() :-
==========

=> function used to extract year / month / day from date

Ex :-

SQL>SELECT EXTRACT(year from sysdate) ; => 2026
SQL>SELECT EXTRACT(month from sysdate) ; => 10
SQL>SELECT EXTRACT(day from sysdate) ; => 7

=> employees joined in 1981 ?

SQL>SELECT * FROM emp WHERE hiredate BETWEEN '01-JAN-1981' AND '31-DEC-1981' ;

SQL>SELECT * FROM emp WHERE EXTRACT(year from hiredate) = 1981;

=> employees joined in 1980,1983,1985 ?

SQL>SELECT * FROM emp WHERE EXTRACT(year from hiredate) IN (1980,1983,1985) ;

=> employees joined in leap year ?

SQL>SELECT * FROM emp WHERE MOD(EXTRACT(year from hiredate) ,4) = 0 ;

=> employees joined jan,apr,dec months ?

SQL>SELECT * FROM emp WHERE EXTRACT(month from hiredate) IN (1,4,12) ;

ROUND & TRUNC :-
==========

=> these two functions can also be used to round dates
=> dates can be rounded to year / month / day

Ex :-

SQL>SELECT ROUND(SYSDATE,'YEAR'); => 01-JAN-27

01-JAN-2026----------30-JUN----------01-JAN-2027

SQL>SELECT ROUND(SYSDATE,'MONTH'); => 01-OCT-26

01-OCT-26----------15-OCT----------01-NOV-26

SQL>SELECT ROUND(SYSDATE,'DAY'); =>

4-OCT-26----------WED----------11-OCT-26

SQL>SELECT ROUND(SYSDATE); => 08-OCT-26

07-OCT-26----------12PM----------08-OCT-26

SQL>SELECT TRUNC(SYSDATE,'YEAR'); => 01-JAN-26
SQL>SELECT TRUNC(SYSDATE,'MONTH'); => 01-OCT-26
SQL>SELECT TRUNC(SYSDATE,'DAY'); => 4-OCT-26
SQL>SELECT TRUNC(SYSDATE); => 7-OCT-26

Ex :-

step 1 :- insert new employee data

SQL>INSERT INTO emp (empno,ename,sal,hiredate) VALUES(100,'ABC',2000,SYSDATE);

step 2 :- list of employees joined today ?

SQL>SELECT * FROM emp WHERE hiredate = SYSDATE ; => no rows

07-OCT-26 12:10:57 = 07-OCT-26 12:39:00

"=" comparison with SYSDATE always fails because oracle not only compares date but also
compares time , to overcome this problem apply TRUNC function on both sides

SQL>SELECT * FROM emp WHERE TRUNC(hiredate) = TRUNC(SYSDATE);

07-oct-26 = 07-oct-26

8-OCT-26

ADD_MONTHS() :-
==========

=> used to add / subtract months to / from a date

ADD_MONTHS(date , months)

Ex :-

SQL>SELECT ADD_MONTHS(SYSDATE,2) ; => 8-DEC-26
SQL>SELECT ADD_MONTHS(SYSDATE,-2) ; => 8-AUG-26

=> employees joined in last 6 months ?

SQL>SELECT *
FROM emp
WHERE hiredate BETWEEN ADD_MONTHS(SYSDATE,-6) AND SYSDATE;

WHERE hiredate >= ADD_MONTHS(SYSDATE,-6) ;

=>

gold_rates
dateid rate
01-jan-20 ?
02-jan-20 ?

08-oct-26 ?

=> display today's gold rate ?

```
SELECT *  FROM  gold_rates WHERE  TRUNC(dateid) =  TRUNC(SYSDATE) ;
```

=> display yesterday's gold rate ?

```
SELECT *  FROM  gold_rates WHERE  TRUNC(dateid) =  TRUNC(SYSDATE-1) ;
```

=> display last month same day gold rate ?

```
SELECT *
FROM  gold_rates
WHERE  TRUNC(dateid) =  TRUNC( ADD_MONTHS(SYSDATE,-1));
```

=> displlay last year same day gold rate ?

```
SELECT *
FROM  gold_rates
WHERE  TRUNC(dateid) =  TRUNC(ADD_MONTHS(SYSDATE,-12));
```

=> display last 1 month gold rates ?

```
SELECT *
FROM  gold_rates
WHERE  TRUNC(dateid) BETWEEN  TRUNC(ADD_MONTHS(SYSDATE,-1))  AND
TRUNC(SYSDATE) ;
```

MONTHS_BETWEEN() :-
==========

=> returns no of months between two dates

MONTHS_BETWEEN(DATE1,DATE2)

Ex :-

```
SQL>SELECT  MONTHS_BETWEEN(SYSDATE,'08-OCT-25') ;    => 12
```

=> display ENAME EXPERIENCE in months ?

```
SQL>SELECT ename, FLOOR(MONTHS_BETWEEN(SYSDATE,hiredate))  as expr  FROM
```

emp ;

smith 549
allen 547

=> display ENAME EXPERIENCE ?
M YEARS N MONTHS

experience = 40 months = 3 YEARS 4 MONTHS

YEARS = MONTHS/12 = 40/12 = 3

MONTHS = MOD(MONTHS,12) = MOD(40,12) = 4

```
SQL>SELECT  ename ,
                FLOOR(MONTHS_BETWEEN(SYSDATE,hiredate)/12)  AS YEARS,
                MOD(FLOOR(MONTHS_BETWEEN(SYSDATE,hiredate)),12) AS MONTHS
      FROM emp ;
```

smith 45 9
allen 45 7

17-DEC-1980 17-DEC-2025 45
17-SEP-2026 9

=> display ENAME EXPERIENCE ?
M YEARS N MONTHS D DAYS

LAST_DAY() :-
==========

=> returns last day of the month

LAST_DAY(DATE)

Ex :-

SQL>SELECT LAST_DAY(SYSDATE) ; => 31-OCT-26

=> display next month 1st day ?

SQL> SELECT LAST_DAY(SYSDATE) + 1 ;

=> display current month 1st day ?

SQL> SELECT ADD_MONTHS (LAST_DAY(SYSDATE) + 1 ,-1) ;

NEXT_DAY() :-
==========

=> returns next specified day starting from given date

NEXT_DAY(DATE,DAY)

Ex :-

SQL>SELECT NEXT_DAY(SYSDATE,'SUNDAY') ; => 11-OCT-26

1 display first sunday of the next month ?
2 display last sunday of current month ?

Conversion Functions :-
==========

=> conversion means converting from one datatype to another datatype
=> conversion is 2 types

1 implicit conversion
2 explicit conversion

implicit conversion :-
==========

=> if conversion performed by oracle then it is called implicit conversion

Ex 1 :-

SQL>SELECT 1000 + '5000' ; => 6000

string '5000' converted to number by oracle

Ex 2 :-

SQL>CREATE TABLE D(X DATE);

SQL>INSERT INTO D VALUES('08-OCT-26'); => 1 row created

string '08-OCT-26' converted to date by oracle

NOTE :-

=> implicit conversion is not possible always some times fails
=> oracle doesn' t recommed implicit conversion because implicit conversion degrades performance

9-oct-26

Explicit conversion :-
==========

=> conversion performed by user is called explicit conversion
=> the following functions provided by oracle for explicit conversion

1 TO_CHAR
2 TO_DATE
3 TO_NUMBER

converting date to char type :-
==========

=> dates converted to char type to display dates in different formats

TO_CHAR(DATE , 'FORMAT') DATE => SYSDATE

FORMATS :-
==========

YYYY	2026
YY	26
YEAR	TWENTY TWENTY-SIX
MM	10
MON	OCT

MONTH	OCTOBER
DDD	281 (day of the year)
DD	08
D	5
DY	THU
DAY	THURSDAY
HH	hour part
HH24	hour part in 24 hrs format
MI	minutes
SS	seconds
AM/PM	AM time / PM time
Q	quarter (1-4)
W	week of the month
WW	week of the year
fm	format mask => removes extra spaces

ex :-

SQL>SELECT TO_CHAR(SYSDATE,'yyyy yy year'); => 2026 26 twenty twenty-six

SQL>SELECT TO_CHAR(SYSDATE,'mm mon month'); => 10 oct october

SQL>SELECT TO_CHAR(SYSDATE,'ddd dd d dy day'); => 282 09 6 fri friday

SQL>SELECT TO_CHAR(SYSDATE,'hh hh24 mi ss'); => 11 11 29 59

SQL>SELECT TO_CHAR(SYSDATE,'Q'); => 4

SQL>SELECT TO_CHAR(SYSDATE,'W WW'); => 2 41

=> display ENAME JOIN_DAY ?

SQL>SELECT ename,TO_CHAR(hiredate,'day') as day FROM emp ;

=> employees joined on sunday ?

SQL>SELECT * FROM emp WHERE TO_CHAR(hiredate,'dy') = 'sun' ;

```
SQL>SELECT *  FROM emp WHERE TO_CHAR(hiredate,'d') = 1 ;

SQL>SELECT *
        FROM emp
        WHERE  TO_CHAR(hiredate,'fmday') = 'sunday' ;  => no rows
```

=> display employees joined in 2nd quarter of 1981 year ?

```
SQL>SELECT *
        FROM emp
        WHERE  TO_CHAR(hiredate,'yyyy') = 1981
                    AND
                    TO_CHAR(hiredate,'q') = 2 ;

        WHERE TO_CHAR(hiredate,'yyyy-q')  =  '1981-2' ;
```

=> display current date & time ?

```
SQL>SELECT TO_CHAR(SYSDATE,'yyyy-mm-dd hh:mi:ss PM') ;
```

=> display ENAME HIREDATE ?
display hiredate in mm/dd/yyyy format ?

```
SQL>SELECT ename,TO_CHAR(hiredate,'mm/dd/yyyy') as hire  FROM emp  ;
```

converting number to char type :-
==========

=> numbers converted to char type to display numbers in different formats

TO_CHAR(number,'format')

formats :-
=======

9	represents a digit
0	represents a digit
G	thousand seperator
D	decimal seperator
L	currency symbol
C	currency

Ex :-

```
SQL>SELECT  TO_CHAR(1234,'99999');   =>       1234
SQL>SELECT TO_CHAR(1234,'00000');            =>       01234
SQL>SELECT TO_CHAR(1234,'000000');  =>       001234
SQL>SELECT TO_CHAR(1234,'9G999');   =>       1,234
SQL>SELECT  TO_CHAR(500000,'9G99G999');   =>       5,00,000
SQL>SELECT TO_CHAR(5000,'9G999D99');       =>       5,000.00
SQL>SELECT TO_CHAR(5000,'L9G999'); =>       $5,000
SQL>SELECT TO_CHAR(5000,'C9G999');=>       USD5,000
```

=> display ENAME   SAL   ?
    display salaries with thousand seperator and currency symbol ?

```
SQL>SELECT ename , TO_CHAR(sal , 'c99G999')  as sal FROM emp ;
```

how to change currency :-
==========

```
SQL>ALTER SESSION SET NLS_TERRITORY = 'INDIA' ;
```

NLS => National Language Specification

Converting date string to date :-
==========

Date string => '09-oct-26'
'10/09/26'
'2026-10-09'

TO_DATE (date string , 'format')

Ex :-

```
SQL>SELECT SYSDATE + 10 ;     =>       19-OCT-26

SQL>SELECT '01-JAN-27' + 100;   =>       ERROR

SQL>SELECT TO_DATE('01-JAN-27','DD-MON-YY')   + 100;  => 11-APR-27
```

calculate '01/01/27' + 100 ?

```
SQL>SELECT TO_DATE('01/01/27','MM/DD/YY') + 100 ;   => 11-APR-27
```

=> waq to display on which day india got independence ?

SQL>SELECT TO_CHAR(TO_DATE('15-AUG-1947','DD-MON-YYYY'),'DAY') ; => FRIDAY

10-oct-26

converting string to number :-
==========

TO_NUMBER(string,'format')

Ex :-

SQL>SELECT 5000 + '1000' ; => 6000 (implicit conversion)

SQL>SELECT 5000 + '1,000' ; => ERROR (implicit conversion fails)

SQL>SELECT 5000 + TO_NUMBER('1,000','9G999') ; => 6000

=> calculate '$5,000' + 'USD1,000' ?

SQL>SELECT TO_NUMBER('$5,000','L9G999') + TO_NUMBER('USD1,000','C9G999') ;

o/p :- 6000

INPUT	OUTPUT	FUNCTION
DD-MON-YY	MM/DD/YY	TO_CHAR
MM/DD/YY	DD-MON-YY	TO_DATE
DEFAULT	OTHER	TO_CHAR
OTHER	DEFAULT	TO_DATE
5000	$5,000	TO_CHAR
$5,000	5000	TO_NUMBER

Special functions :-
==========

NVL() :-
=======

=> used to convert null values

NVL(arg1,arg2)

=> if arg1 = null returns arg2
=> if arg1 <> null returns arg1 only

Ex :-

SQL>SELECT NVL(100,200); => 100
SQL>SELECT NVL(NULL,200); => 200

=> display ENAME SAL COMM TOTSAL ?

totsal = sal + comm

SQL>SELECT ename,sal,comm , sal + comm as totsal FROM emp ;

SMITH 800 null null
ALLEN 1600 300 1900

x + null => null
x - null => null
x * null => null

SQL> SELECT ename,sal,comm , sal + NVL(comm,0) as totsal FROM emp ;

SMITH 800 NULL 800
ALLEN 1600 300 1900

=> DISPLAY ENAME SAL COMM ?
if comm = NULL display N/A

SQL>SELECT ename,sal,NVL(comm,'N/A') as comm FROM emp ; => ERROR

SQL>SELECT ename,sal,NVL(TO_CHAR(comm),'N/A') as comm FROM emp ;

=> display SNO M P C TOTAL AVG ?
if marks = null display ABSENT ?

STUDENT

sno	sname	m	p	c
1	A	80	90	70
2	B	NULL	60	50

```
 3      C        50        NULL   NULL
```

```
 SQL>SELECT  sno , NVL(TO_CHAR(m) , 'ABSENT')  as  m ,
                          NVL(TO_CHAR(p), 'ABSENT') as p,
                          NVL(TO_CHAR(c),'ABSENT') as c,
                          NVL(m,0) + NVL(p,0) + NVL(c,0) as total,
                          (NVL(m,0) + NVL(p,0) + NVL(c,0))/3 as avg
        FROM student;
```

COALESCE :-
==========

=> returns first not null expression

```
          COALESCE(arg1,arg2,arg3,-----------------)
```

Ex :-

```
 SQL>SELECT  COALESCE(NULL,100,NULL,200) ;    => 100
 SQL>SELECT  COALESCE(100,200,300) ;          => 100
```

=> DISPLAY cid cname best_contact ?

best_contact is phone
if phone is absent emailid
if emailis also absent then addr

```
 cust
 cid    cname addr    emailid          phone
 100    A       HYD    abc@gmail.com        123
 101    B       BLR    xyz@gmail.com        NULL
 102    C       MUM    NULL             NULL
```

```
 SQL>SELECT  cid,cname,
                    COALESCE(phone,emailid,addr) as best_contact
        FROM cust;

         100    A       123
         101    B       xyz@gmail.com
         102    C       MUM
```

How to handle nulls in oracle ?

ans :- using NVL,COALESCE functions

ASCII() :-
=======

=> returns ascii value of a given char

ASCII(char)

Ex :-

SQL>SELECT ASCII('A') ; => 65

CHR() :-
=======

=> returns char for given ascii value

CHR(ascii value)

Ex :-

SQL>SELECT CHR(65) ;

SQL> SELECT 'A'||CHR(10)||'B' ; => A
B

=> display

cname
hno
street
city
state - pin

CUST

cid	cname	hno	street	city	state	pin ?
100	A	100A	AMPT	HYD	TG	99999

```
SQL>SELECT  cname||CHR(10)||
            hno||CHR(10)||
            street||CHR(10)||
            city||CHR(10)||
            state||'-'||pin
```

```
          FROM cust
           WHERE cid = 100 ;
```

12-oct-26

How to implement if-else in SQL :-
==========

1 DECODE function
2 CASE statement

CASE statement :-
==========

=> used to implement IF-THEN-ELSE
=> using case statement we can return values based on a condition
=> case statements are 2 types

1 simple case
2 searched case

simple case :-
==========

=> use simple case when condition based on "=" operator

```
CASE colname
WHEN value1 THEN return expr1
WHEN value2 THEN return expr2
 ----------------------------------------
[ ELSE return expr]
END
```

Ex :-

=> display ENAME   JOB   ?

```
        if JOB=CLERK display        WORKER
                SALESMAN display  EXECUTIVE
                MANAGER             BOSS
                PRESIDENT           BIG BOSS
```

```
 SQL>SELECT ename,
```

```
                CASE job
                WHEN 'CLERK'  THEN  'WORKER'
                WHEN 'SALESMAN' THEN 'EXECUTIVE'
                WHEN 'MANAGER'  THEN 'BOSS'
                WHEN 'PRESIDENT'  THEN 'BIG BOSS'
                ELSE 'EMPLOYEE'
                END  as job
        FROM emp ;
```

=> increment salaries as follows ?

```
     if deptno=10  incr  sal by 10%
                20                15%
                30                20%
               others             5%

    SQL>UPDATE emp
          SET sal  = CASE deptno
                     WHEN 10 THEN  sal*1.1
                     WHEN 20 THEN sal*1.15
                     WHEN 30 THEN sal*1.2
                     ELSE sal*1.05
                     END ;
```

searched case :-
==========

=> use searched case when conditions not based on "=" operator and based on > < between etc

```
CASE
WHEN cond1 THEN return expr1
WHEN cond2 THEN return expr2
---------------------
ELSE return expr
END
```

Ex :-

=> display ENAME SAL SALRANGE ?

```
              IF  sal>3000  display  HISAL
```

```
                sal<3000              LOSAL
                sal=3000            AVGSAL

  SQL>SELECT ename,sal,
                  CASE
                  WHEN sal>3000 THEN 'HISAL'
                  WHEN sal<3000  THEN 'LOSAL'
                  ELSE 'AVGSAL'
                  END as salragne
          FROM emp ;


 UPDATE cust
 SET gender =  CASE
                WHEN SUBSTR(LTRIM(gender),1,1)  IN ('m','M') THEN 'MALE'
                WHEN  SUBSTR(LTRIM(gender),1,1) IN ('f','F') THEN 'FEMALE'
                ELSE 'unknown'
                 END;

 Analytical Functions in  oralce :-
=========================


 RANK & DENSE_RANK :-
 ====================

 => both functions used to find ranks

        RANK() OVER (ORDER BY colname ASC/DESC)
        DENSE_RANK() OVER (ORDER BY colname ASC/DESC)

Ex :-

=> find ranks of the employees based on sal  highest paid should get 1st rank ?

   SQL>SELECT ename,sal,
                  RANK() OVER (ORDER BY sal DESC) as rnk
          FROM emp ;

  SQL>SELECT ename,sal,
                  DENSE_RANK() OVER (ORDER BY sal DESC) as rnk
          FROM emp ;
```

difference between rank & dense_rank ?

=> if there is a tie then rank function will skip next rank but dense_rank will not skip

SAL	RANK	DENSE_RANK
5000	1	1
4000	2	2
3000	3	3
3000	3	3
3000	3	3
2000	6	4
2000	6	4
1000	8	5

Multif-row functions :-
==========

=> single row function returns one value per row and multi row functions return one value from group rows

=> multi-row functions are also called aggregate functions or group functions

MAX
MIN
SUM
AVG
COUNT
COUNT(*)

MAX() :-
========

=> returns maximum value

MAX(arg)

SQL>SELECT MAX(sal) FROM emp ; => 5000
SQL>SELECT MAX(ename) FROM emp; => ward
SQL>SELECT MAX(hiredate) FROM emp; => 12-JAN-83

MIN() :-
======

=> returns minimum value

MIN(arg)

SQL>SELECT MIN(sal) FROM emp ; => 800

SUM() :-
=======

=> returns total

SUM(arg)

Ex :-

SQL>SELECT SUM(sal) FROM emp ; => 29025

=> round total sal to hundreds ?

SQL>SELECT ROUND(SUM(sal) , -2) FROM emp ; => 29000

29000----------29050 ----------29100

=> after rounding display total sal with thousand seperator and currrency symbol ?

SQL>SELECT TO_CHAR(ROUND(SUM(sal) , -2) , 'L99G999') FROM emp ; => $29,000

AVG() :-
=======

=> returns average value

AVG(arg)

Ex :-

SQL>SELECT AVG(sal) FROM emp ; => 2073.21429

=> round avg sal to lowest ?

SQL>SELECT FLOOR(AVG(sal)) FROM emp ; => 2073

=> SUM,AVG functions cannot be applied on char,date columns

COUNT() :-
========

=> returns no of values present in a column

COUNT(colname)

Ex :-

SQL>SELECT COUNT(empno) FROM emp ; => 14
SQL>SELECT COUNT(comm) FROM emp ; => 4

COUNT(*) :-
=========

=> returns no of rows in a table

SQL>SELECT COUNT(*) FROM emp ; => 14

difference between COUNT & COUNT(*) ?

	COUNT	COUNT(*)
1	reutrns no of values present in a column	returns no of rows in a table
2	doesn't count nulls	counts nulls

Ex :-

T1
F1
10
NULL
20
NULL
30

COUNT(F1) => 3

COUNT(*) => 5

=> display no of employees joined on sunday ?

```
SQL>SELECT COUNT(*) FROM emp WHERE TO_CHAR(hiredate,'dy') = 'sun' ;
```

=> no of employees joined in 2nd quarter of 1981 year ?

```
SQL>SELECT COUNT(*)
        FROM emp
       WHERE TO_CHAR(hiredate,'q') = 2  AND TO_CHAR(hiredate,'yyyy') = 1981;
```

NOTE :-

=> aggregate functions are not allowed in where clause and they are allowed only in SELECT,HAVING clauses.

```
SELECT ename
FROM emp
WHERE sal = MAX(sal);  => ERROR
```

How to use aggregate functions as analytical function :-

==========

=> analytical function returns one value per row but aggregate functions returns one value from group of rows.

```
SUM(colname) OVER (ORDER BY colname ----)
```

Ex 1 :-

```
SQL>SELECT empno,ename,sal,
                SUM(sal) OVER (ORDER BY empno ASC)  as running_total
        FROM emp ;
```

Ex 2 :-

TRANSACTIONS

TRID		TTYPE	TDATE	TAMT	ACCNO
1	W	?	2000	100	
2	D		5000	100	
3	W		3000	100	

4 D 8000 100

=> display TDATE TAMT RUNNING_BALANCE ?

```
SELECT  TRID,TDATE,TAMT,
        SUM(CASE TTYPE
            WHEN 'W' THEN -TAMT
            WHEN 'D' THEN  TAMT
            END) OVER (ORDER BY TDATE ASC) AS RUNNING BALANCE
 FROM TRANSACTIONS ;
```

1	W	2000	-2000
2	D	5000	3000
3	W	3000	0
4	D	8000	8000

summary :-

char :- upper,lower,initcap,length,substr,instr,lpad,rpad,ltrim,rtrim,trim,replace,translate
numeric :- abs,power,sqrt,mod,round,trunc,ceil,floor
date :- extract,add_months,months_between,last_day,next_day
conversion :- to_char,to_date,to_number
special :- nvl,coalesce,ascii,chr
analytical :- rank,dense_rank
aggregate :- max,min,sum,avg,count,count(*)

==========

GROUP BY clause :-
==========

=> GROUP BY clause is used to group rows based on one or more columns to calculate max,min,sum,avg,count for each group. For ex to calculate dept wise total sal first group the rows based on dept and apply sum function on each dept.

EMP

empno	ename	sal	deptno

1	A	3000	10				
2	B	5000	20	GROUP BY	10	8000	
3	C	6000	30	==========>	20	9000	
4	D	5000	10		30	6000	
5	E	4000	20				

detailed data summarized data

=> GROUP BY clause converts detailed data into summarized data which is useful for analysis

```
SELECT columns
FROM  tabname
[WHERE cond]
GROUP BY col1,col2,-----
[HAVING cond]
[ORDER BY ----] ;
```

EXECUTION :-
==========

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
'

Ex :-

=> display dept wise total salaries ?

```
SELECT deptno,SUM(sal)
FROM emp
GROUP BY deptno ;
```

FROM emp :-
==========

empno	ename	sal	deptno
1	A	3000	10
2	B	5000	20
3	C	6000	30
4	D	5000	10

```
    5     E      4000   20

 GROUP BY deptno :-
 ================

10      1       A       3000
        4       D       5000


20      2       B       5000
        5       E       4000

30      3       C       6000

 SELECT deptno,SUM(sal) :-
 ======================

        10      8000
        20      9000
        30      6000
```

=> display job wise summary ?

```
    SELECT job,MIN(sal) as minsal,
                MAX(sal) as maxsal,
                SUM(sal) as totsal,
                AVG(sal) as avgsal,
                COUNT(*) as cnt
    FROM emp
    GROUP BY job;
```

```
JOB             MINSAL     MAXSAL     TOTSAL     AVGSAL       CNT
---------               ---------- ---------- ---------- ---------- ----------
CLERK           800       1300       4150     1037.5          4
SALESMAN                1250       1600       5600       1400         4
MANAGER                 2450       2975       8275 2758.33333          3
ANALYST                 3000       3000       6000       3000         2
PRESIDENT               5000       5000       5000       5000         1
```

=> display year wise no of employees joined ?

```
    SELECT TO_CHAR(hiredate,'yyyy') as year,COUNT(*) as cnt
    FROM emp
```

```
   GROUP BY  TO_CHAR(hiredate,'yyyy') ;

YEAR        CNT
---- -     ---------
1980          1
1981         10
1982          2
1983          1
```

=> display month wise no of employees joined in the year 1981 ?

```
   SELECT TO_CHAR(hiredate,'mon') as mon,COUNT(*) as cnt
   FROM emp
   WHERE TO_CHAR(hiredate,'yyyy') = 1981
   GROUP BY  TO_CHAR(hiredate,'mon') ;
```

=> find the departments having more than 3 employees ?

```
  SELECT deptno,COUNT(*)
  FROM emp
  WHERE  COUNT(*) > 3
  GROUP BY deptno  ;          => ERROR
```

oracle cannot calculate dept wise count before group and it can calculate only after group by, so apply the condition COUNT(*) > 3 after group by using HAVING clause.

```
  SELECT deptno,COUNT(*)
  FROM emp
  GROUP BY deptno
  HAVING COUNT(*) > 3 ;

        20       5
        30       6
```

WHERE VS HAVING :-

==========

	WHERE	HAVING
1	selects specific rows	select specific groups
2	conditions applied before group by	conditions applied after group by

3	use where clause if cond doesn't contain aggregate function.	use having clause if cond contains aggregate function

=> display dept wise no of employees
for only the dept 10,20 and no of employees > 3 ?

```
1    SELECT deptno,COUNT(*)
     FROM emp
     WHERE deptno IN (10,20)
     GROUP BY deptno
     HAVING COUNT(*) > 3 ;

         20       5
```

```
2    SELECT deptno,COUNT(*)
     FROM emp
     GROUP BY deptno
     HAVING COUNT(*) > 3  AND deptno IN (10,20)  ;
```

query 1 gives good performance than query2
for better performance filter data before group by

=> list of customers placed more than 5 orders in last 6 months ?

```
ORDERS
ordid          orddt    deldt    cid

SELECT cid,COUNT(*)
FROM orders
WHERE  orddt BETWEEN  ADD_MONTHS(SYSDATE,-6)  AND  SYSDATE ;
GROUP BY cid
HAVING COUNT(*) > 5 ;
```

=> display southern states having more than 5cr population ?

```
PERSONS
aadharno    name   gender dob      addr     city      state

 SELECT state,COUNT(*)
 FROM persons
```

```
    WHERE state IN ('AP','TG','KA','TN','KL')
    GROUP BY state
     HAVING  COUNT(*) > 50000000 ;
```

Grouping based on multiple fields :-
==========

=> display dept wise and with in dept job wise total salary ?

```
 SQL>BREAK ON DEPTNO SKIP 1

  SQL>SELECT deptno,job,SUM(sal)
   FROM emp
   GROUP BY deptno,job
   ORDER BY deptno ASC ;

        10      CLERK               1300
        10      MANAGER     2450
        10      PRESIDENT   5000

        20      ANALYST             6000
        20      CLERK               1900
        20      MANAGER     2975

        30      CLERK               950
        30      MANAGER     2850
        30      SALESMAN    5600
```

=> display year wise and with in year quarter wise no of employees joined ?

```
        1981   1        ?
               2        ?
               3        ?
               4        ?

     SQL>SELECT TO_CHAR(hiredate,'yyyy') as year,
                TO_CHAR(hiredate,'q') as qrt,
                COUNT(*) as cnt
          FROM emp
          GROUP BY  TO_CHAR(hiredate,'yyyy'),TO_CHAR(hiredate,'q');
```

ROLLUP & CUBE :-
==========

=> both functions used to find subtotals and grand total

```
GROUP BY ROLLUP(col1,col2,---)
GROUP BY CUBE(col1,col2,---)
```

ROLLUP :-
=========

=> ROLLUP displays subtotals for each group and also displays grand total

Ex :-

```
SELECT deptno,job,SUM(sal)
FROM emp
GROUP BY ROLLUP(deptno,job)
ORDER BY deptno ASC ;
```

```
    10  CLERK              1300
10      MANAGER    2450
10      PRESIDENT  5000
                   8750   => dept subtotal

20      ANALYST            6000
20      CLERK              1900
20      MANAGER    2975
                   10875 => dept subtotal

30      CLERK              950
30      MANAGER    2850
30      SALESMAN   5600
                   9400  => dept subtotal

                   29025 => grand total
```

CUBE :-
=======

=> CUBE displays subtotals for each group by column (deptno,job) and also displays grand total

```
SELECT deptno,job,SUM(sal)
FROM emp
GROUP BY CUBE(deptno,job)
```

ORDER BY deptno ASC ;

```
 10      CLERK                1300
10       MANAGER     2450
10       PRESIDENT   5000
                     8750   => dept subtotal

20       ANALYST              6000
20       CLERK                1900
20       MANAGER     2975
                     10875 => dept subtotal

30       CLERK                950
30       MANAGER     2850
30       SALESMAN    5600
                     9400  => dept subtotal

          ANALYST              6000  => job subtotal
          CLERK                4150
          MANAGER               8275
          PRESIDENT            5000
          SALESMAN    5600 => job subtotal

                               29025   => grand total
```

summary :-

importance of group by
writing queries using group by
where vs having
rollup & cube

==========
===

15-OCT-26

INTEGRITY CONSTRAINTS :-
==========

=> constraints are rules enforced on columns to ensure data accuracy or
data quality or data integrity.

=> integrity constraints are used to prevent users from entering invalid data.

Types of constraints :-
==========

1 NOT NULL
2 UNIQUE
3 PRIMARY KEY
4 CHECK
5 FOREIGN KEY
6 DEFAULT

=> above constraints can be declared in two ways

1 column level
2 table level

column level :-
==========

=> if constraints are declared immediately after declaring column then it is called column level

```
create table  <tabname>
(
    colname  datatype(size)  constraint ,
    colname  datatype(size)  constraint,
    ----------
 );
```

NOT NULL :-
==========

=> NOT NULL constraint doesn't accept null values

```
CREATE TABLE emp11
(
   empno  NUMBER(4),
   ename  VARCHAR2(10) NOT NULL
);
```

INSERT INTO emp11 VALUES(100,''); => ERROR

```
 INSERT INTO emp11 VALUES(101,'A');
```

UNIQUE :-
==========

=> unique constraint doesn't accept duplicates

```
   CREATE TABLE cust
   (
      custid  NUMBER(3),
      cname  VARCHAR2(10) NOT NULL,
      emailid  VARCHAR2(20) UNIQUE
     );

INSERT INTO cust VALUES(100,'A','abc@gmail.com');
INSERT INTO cust VALUES(101,'B','abc@gmail.com');  => ERROR
INSERT INTO cust VALUES(102,'C','');
INSERT INTO cust VALUES(103,'D','');
```

NOTE :- unique constraint allows nulls

PRIMARY KEY :-
==============

=> primary key doesn't accept duplicates and nulls
=> it is the combination of unique and not null

primary key = unique + not null

=> In tables one column must be there to uniquely identify and into that column duplicates and nulls are not allowed , so declare that column with primary key.

```
  CREATE TABLE emp15
  (
      empno  NUMBER(4)   PRIMARY KEY,
      ename   VARCHAR2(10) NOT NULL
   );

  INSERT INTO emp15 VALUES(100,'A');
  INSERT INTO emp15 VALUES(100,'B');   => ERROR
  INSERT INTO emp15 VALUES(NULL,'B');  => ERROR
```

NOTE :-

=> a table can have only one primary key , if we want multiple primary keys then declare one column with primary key and other columns with UNIQUE NOT NULL.

Ex :-

```
CREATE TABLE cust
(
    custid  NUMBER(6) PRIMARY KEY,
    cname VARCHAR2(10) NOT NULL,
    aadharno NUMBER(12)  UNIQUE NOT NULL,
    panno   CHAR(10) UNIQUE NOT NULL
);
```

=> columns declared with UNIQUE NOT NULL are called secondary keys

difference between UNIQUE & PRIMARY KEY ?

	UNIQUE	PRIMARY KEY
1	allows nulls	doesn't allow nulls
2	can have multiple unique constraints	only one primary key is allowed per table

CHECK :-
========

=> use check constrain when rule based on condiiton

```
CHECK(condition)
```

Ex 1 :- sal must be min 3000

```
CREATE TABLE emp17
(
    empno  NUMBER(4) PRIMARY KEY,
    ename  VARCHAR2(10) NOT NULL,
    sal       NUMBER(7)  CHECK(sal>=3000)
);

INSERT INTO emp17 VALUES(100,'A',1000);  => ERROR
INSERT INTO emp17 VALUES(101,'B',6000);
INSERT INTO emp17 VALUES(102,'C',NULL);
```

NOTE :- check constraint allows nulls

Ex 2 :- GENDER must be 'M','F' ?

```
GENDER    CHAR(1)   CHECK(GENDER IN ('M','F'))
```

Ex 3 :- AMT mut be multiple of 100

```
AMT     NUMBER(8)   CHECK(MOD(AMT,100)=0)
```

Ex 4 :- PWD must be min 6 chars

```
PWD   VARCHAR2(12)  CHECK( LENGTH(PWD) >= 6)
```

Ex 5 :- EMAILID must contain '@'
must ends with '.com' or '.co' or '.in'

```
EMAILID  VARCHAR2(30)  CHECK(EMAILID LIKE  '%@%'
                                AND
                                (
                                EMAILID LIKE  '%.com'
                                OR
                                EMAILID LIKE '%.co'
                                OR
                                EMAILID  LIKE '%.in'
                                 ))
```

16-oct-26

FOREIGN KEY :-
==========

=> foreign key is used to establish relationship between two tables.

=> To establish relationship take primary key of one table and add it to another table as foreign key and declare with references constraint.

PROJECTS

projid	pname	duration	cost	client
1000	AAA	5	800	TATA MOTORS
1001	BBB	3	300	DBS Bank

EMP

empid	ename	job	sal	projid	REFERENCES projects(projid)
100	A	SE	40K	1000	
101	B	SSE	90K	1001	
102	C	TL	120K	9999 => error	
103	D	SE	30K	1000	
104	E	SE	25K	NULL	

=> values entered in fk column should match with values entered in pk column
=> fk allows duplicates & nulls
=> after declaring fk a relationship is created between two tables and that relationship is called parent / child relationship.
=> pk table is parent and fk table is child

```
CREATE TABLE projects
(
  projid NUMBER(4) PRIMARY KEY,
  pname VARCHAR2(10) NOT NULL
);

INSERT INTO projects VALUES(1000,'A'),(1001,'B');

CREATE TABLE emp_proj
(
  empid  NUMBER(3) PRIMARY KEY,
  ename VARCHAR2(10) NOT NULL,
  sal       NUMBER(7) CHECK(sal>=5000),
  projid   NUMBER(4)  REFERENCES projects(projid)
);

INSERT INTO emp_proj VALUES(100,'ABC',5000,1000);
INSERT INTO emp_proj VALUES(101,'XYZ',6000,9999); => ERROR
INSERT INTO emp_proj VALUES(102,'KLM',6000,1000);
INSERT INTO emp_proj VALUES(103,'DDD',6000,NULL);
```

Relationship Types :-
==========

1 one to one (1:1)
2 one to many (1:m)
3 many to one (m:1)
4 many to many (m:n)

=> by default oracle creates one to many relationship between two tables

How to establish one to one relationship :-
==========

=> To establish one to one relationship between two tables declare fk with unique constraint

ex :-

PERSON
AADHARNO NAME DOB

PASSPORT
PASSPORTNO DOI DOE AADHARNO REFERENCES PERSONS(AADHARNO) UNIQUE

How to establish many to many relationship :-
==========

=> To establish many to many relationship between two tables , create 3rd table
and add primary keys of both tables as foreign keys.

ex :-

COURSE STUDENT

CID	CNAME	SID	SNAME
10	JAVA	1	A
11	ORACLE	2	B

REGISTRATIONS

SID	CID	DOR	FEE
1	10	?	10000
1	11	?	5000
2	10	?	10000
2	11	?	50000

DEFAULT :-
==========

=> a column can be declared with default value as follows

ex :- hiredate date default sysdate

=> while inserting if we skip hiredate then oracle inserts default value i.e. sysdate

```
CREATE TABLE emp22
(
 empno NUMBER(4) PRIMARY KEY,
 ename VARCHAR2(10) NOT NULL,
 hiredate  DATE  DEFAULT SYSDATE
);

INSERT INTO emp22(empno,ename)  VALUES(100,'A') ;
 INSERT INTO emp22 VALUES(101,'B','20-APR-26');
INSERT INTO emp22 VALUES(102,'C','');

SELECT * FROM emp22;
```

EMPNO	ENAME	HIREDATE
100	A	16-OCT-26
101	B	20-APR-26
102	C	

=> create following tables with given rules ?

ACCOUNTS
accno actype bal

Rules :-
=======

1 accno should not be duplicate and null
2 actype must be 's' or 'c'
3 bal must be min 1000

TRANSACTIONS
trid ttype tdate tamt accno

Rules :-
======

1 trid should not be duplicate & null

2 ttype must be 'W' OR 'D'
3 tdate should be always sysdate
4 tamt must be multiple of 100
5 accno should match with accounts table accno

TABLE LEVEL :-
==========

=> if constraints are declared after declaring all columns then it is called table level
=> use table level to declare constraints for multiple or combination of columns

Declaring check constraint at table level :-
==========

products
prodid pname price mfd_dt exp_dt
100 AAA 50 01-OCT-26 01-SEP-26 => invalid

rule :- exp_dt > mfd_dt

```
CREATE TABLE products
(
 prodid  NUMBER(3) PRIMARY KEY,
 pname VARCHAR2(10) NOT NULL,
 price  NUMBER(5),
 mfd_dt  DATE  ,
 exp_dt  DATE ,
            CHECK(exp_dt > mfd_dt)
 ) ;
```

INSERT INTO products VALUES(100,'A',50,SYSDATE,'01-SEP-26'); => ERROR
INSERT INTO products VALUES(100,'A',50, '01-SEP-26',SYSDATE);

17-oct-26

Composite primary key :-
==========

=> if primary key declared for combination of columns then it is called composite primary key.

=> In some tables it is not possible to uniquely identify the rows using single column and we need combination of columns to uniquely identify , so declare that combination as primary key at table level.

=> In composite primary key combinaiton should not be duplicate

ex :-

```
COURSE              STUDENT
CID     CNAME               SID     SNAME
-------                     -------
10  JAVA            1       A
11      ORACLE              2       B


REGISTRATIONS
SID CID     DOR     FEE
----------------
1    10      ?       10000
1    11      ?       5000
2    10      ?       10000
2    11      ?       50000
```

```
CREATE TABLE course
 (
  cid    NUMBER(2)  PRIMARY KEY,
  cname VARCHAR2(10) NOT NULL
 );

INSERT INTO course VALUES(10,'java'),(11,'oracle') ;

CREATE TABLE student
(
  sid   NUMBER(2) PRIMARY KEY,
  sname VARCHAR2(10) NOT NULL
);

INSERT INTO student VALUES(1,'A') , (2,'B') ;

CREATE TABLE registrations
(
  sid  NUMBER(2)  REFERENCES student(sid) ,
  cid  NUMBER(2)  REFERENCES course(cid),
  dor  DATE,
  fee  NUMBER(6) ,
```

```
        PRIMARY KEY(sid,cid)
  );

 INSERT INTO registrations VALUES(1,10,SYSDATE,10000);
 INSERT INTO registrations VALUES(1,11,SYSDATE,5000);
 INSERT INTO registrations VALUES(2,10,SYSDATE,5000);
 INSERT INTO registrations VALUES(1,10,SYSDATE,5000);  => ERROR
```

Ex 2 :- identity primary key and write create table script ?

SALES

dateid	prodid	custid	qty	amt	
16-oct-26	100	10	1	2000	
16-oct-26	100	11	1	2000	
16-oct-26		101	10	1	1000
17-oct-26		100	10	1	2000

which of the following constraint cannot be declared at table level ?

A UNIQUE
B CHECK
C NOT NULL
D PRIMARY KEY
E FOREIGN KEY

ANS :- C

Which statements are true regarding constraints ?

A a foreign key cannot contain NULL value F
B a column with UNIQUE constraint can contain NULL value T
C a constraint is enforced only for the INSERT operation on a table F
D all constraints can be defined at column level and table level. F

Which CREATE TABLE statement is valid?

A. CREATE TABLE ord_details
(ord_no NUMBER(2) PRIMARY KEY,
item_no NUMBER(3) PRIMARY KEY,
ord_date DATE NOT NULL);

B. CREATE TABLE ord_details

(ord_no NUMBER(2) UNIQUE , NOT NULL,
item_no NUMBER(3),
ord_date DATE DEFAULT SYSDATE NOT NULL);

C. CREATE TABLE ord_details
(ord_no NUMBER(2) ,
item_no NUMBER(3),
ord_date DATE DEFAULT NOT NULL,
  UNIQUE (ord_no),
  PRIMARY KEY (ord_no));

D. CREATE TABLE ord_details
(ord_no NUMBER(2),
item_no NUMBER(3),
ord_date DATE DEFAULT SYSDATE NOT NULL,
  PRIMARY KEY (ord_no, item_no));

 Adding constraints to existing table :-
 ==============================

  => "ALTER" command is used to add constraints to existing table

   CREATE TABLE emp55
   (
     empid NUMBER(4),
     ename VARCHAR2(10) ,
     sal NUMBER(7),
     dno NUMBER(2)
    );

  Adding primary key :-
  ================

   => add primary key to empid ?

     ALTER TABLE emp55
              ADD PRIMARY KEY(empid);

   note :- pk cannot be added to column that already contains duplicates or nulls.

Adding check constraint :-
====================

=> add check constraint with cond sal >= 3000 ?

```
ALTER TABLE emp55
        ADD CHECK(sal>=3000);

ALTER TABLE emp
        ADD CHECK(sal>=3000);  => ERROR  => some salaries are less than 3000
```

=> above command returns error because some salaries are less than 3000 ,
while adding constraint oracle also validating existing data.

NOVALIDATE :-
==========

=> if constraint is added with NOVALIDATE then oracle will not validate existing data
and it validates only new data.

```
ALTER TABLE emp
        ADD CHECK(sal>=3000) NOVALIDATE ;
```

Adding foreign key :-
==========

=> add fk to dno that should match with dept table primary key i.e. deptno ?

```
ALTER TABLE emp55
      ADD FOREIGN KEY(dno)  REFERENCES dept(deptno) ;
```

Changing from NULL to NOT NULL :-
==========

=> modify column ename to not null ?

```
ALTER TABLE emp55
        MODIFY (ename NOT NULL) ;
```

USER_CONSTRAINTS :-
==========

=> system table that stores information about constraints declared in table

CONSTRAINT_NAME

CONSTRAINT_TYPE
TABLE_NAME
SEARCH_CONDITION

=> list of constraints declared in emp55 table ?

```
SELECT  constraint_name , constraint_type , search_condition
FROM user_constraints
WHERE  table_name = 'EMP55' ;
```

```
SYS_C009037        R
SYS_C009033        C   sal>=3000
SYS_C009036        C   "ENAME" IS NOT NULL
SYS_C009032        P
```

Droping constraints :-
==========

```
ALTER TABLE <tabname>
        DROP CONSTRAINT  <name>;
```

Ex :-

=> drop check constraint in emp55 table ?

```
ALTER TABLE emp55
        DROP CONSTRAINT  SYS_C009033
```

=> drop primary key in emp55 table ?

```
ALTER TABLE emp55
        DROP  PRIMARY KEY ;
```

=> pk constraint cannot be dropped if referenced by some fk
pk table cannot be dropped if referenced by some fk
pk table cannot be truncated if referenced by some fk

```
ALTER TABLE dept
        DROP  PRIMARY KEY ;  => error

DROP TABLE dept ;          => error

TRUNCATE TABLE dept;  => error
```

==========

23-oct-26

JOINS :-
========

=> join is an operation performed to display data from two or more table
for ex to display data from two tables we need to join those tables.

=> In db tables are normalized i.e. related data stored in multiple tables ,
To gather or to combine data stored in multiple tables we need to join
those tables.

ex :-

orders

ordid	orddt	deldt	cid
1000	22-	25-	10
1001	23-	28-	11
1002	23-	25-	12

customers

cid	cname	caddr
10	A	HYD
11	B	HYD
12	C	HYD

output :-

ordid	orddt	deldt	cid	cname	addr
1000	22-	25-	10	A	HYD
1001			11	B	HYD

Types of joins :-
=============

1 Inner join
- equi join
- non equi join
- self join

2 outer join
- left outer join
- right outer join
- full outer join

3 cross / cartesian join

Equi join :-
=========

=> equi join is performed between the tables sharing a common field and name of the common field need not to be same and pk-fk relationship is not compulsory.

=> equi join is performed based on the common field with same datatype.

```
SELECT columns
FROM tab1 INNER JOIN tab2
ON join condition ;
```

join condition :-
==========

=> join condition specifies which record of 1st table joined with which record of 2nd table , based on the given join condition oracle joins the records of two tables.

table1.commonfield = table2.commonfield

=> this is called equi join because here join condition is based on "=" operator

ex :-

EMP

empno	ename	sal	deptno
1	A	3000	10
2	B	4000	20
3	C	5000	30
4	D	3000	20
5	E	2000	NULL

DEPT

deptno	dname	loc
10	ACCOUNTS	NEW YORK
20	RESEARCH	
30	SALES	
4O	OPERATIONS	

=> display employee details with dept details ?

```
SELECT empno,ename,sal,dname,loc
FROM emp INNER JOIN dept
ON emp.deptno = dept.deptno ;
```

1	A	3000	ACCOUNTS	NEW YORK
2	B	4000	RESEARCH	
3	C	5000	SALES	
4	D	3000	RESEARCH	

=> In join queries declare table alias and prefix column names with
    table alias for two reasons

    1 to avoid ambiguity
    2 for faster execution

```
SELECT e.empno,e.ename,e.sal,
            d.deptno,d.dname,d.loc
FROM emp e  INNER JOIN dept d
ON e.deptno = d.deptno ;
```

=> display employee details working at NEW YORK loc ?

```
SELECT e.ename,d.dname,d.loc as city
FROM emp e  INNER JOIN dept d
ON e.deptno = d.deptno    /*  join condition */
WHERE d.loc = 'NEW YORK'   /* filter condition */  ;
```

joining more than two tables :-
==========

=> if no of tables increases , no of join conditions also increases
=> to join N tables N-1 join conditions required

```
SELECT columns
FROM tab1
INNER JOIN tab2
ON join cond1
INNER JOIN tab3
ON join cond2
INNER JOIN tab4
ON join cond3
 ----------
 [WHERE cond] ;
```

((tab1 join tab2) join tab3) join tab4

Ex :-

employees departmnets locations countries

employee_id department_id location_id country_id
first_name department_name city country_name

last_name location_id state
department_id country_id

=> display ENAME DNAME CITY STATE COUNTRY ?
---------- ---------- ---------- ---------- ----------
employees departments locations countries

```
SELECT  e.first_name ,
        d.department_name,
        l.city,l.state,
        c.country_name
FROM  employees e
INNER JOIN departments d
ON e.department_id = d.department_id
INNER JOIN locations l
ON d.location_id = l.location_id
INNER JOIN countries c
ON l.country_id = c.country_id ;
```

OUTER JOIN :-
==========

=> inner join returns only matching records but it will not return unmatched records,
to display unmatched records perform outer join.

ex :-

EMP				DEPT		
empno	ename	sal	deptno	deptno	dname	loc
1	A	3000	10	10	ACCOUNTS	NEW YORK
2	B	4000	20	20	RESEARCH	
3	C	5000	30	30	SALES	
4	D	3000	20	4O	OPERATIONS	=> unmatched row
5	E	2000	NULL => unmatched row			

=> outer join is 3 types

1 left outer join
2 right outer join
3 full outer join

LEFT OUTER JOIN :-
==========

=> returns all rows (matched + unmatched) from left side table and matching rows from right side table

```
SELECT e.ename,d.dname
FROM emp e LEFT OUTER JOIN dept d
ON e.deptno = d.deptno ;
```

=> above query returns all rows from emp and matching rows from dept

```
A       ACCOUNTS
B       RESEARCH
C       SALES
D       RESEARCH
E       NULL   => unmatched from emp
```

24-oct-26

RIGHT OUTER JOIN :-
==========

=> returns all rows (matched + unmatched) from right side table and matching rows from left side table.

```
SELECT e.ename,d.dname
FROM emp e  RIGHT OUTER JOIN dept d
ON e.deptno = d.deptno ;
```

=> above query returns all rows from dept table and matching rows from emp

```
A       ACCOUNTS
B       RESEARCH
C       SALES
D       RESEARCH
NULL  OPERATIONS   => unmatched from dept
```

FULL OUTER JOIN :-
==========

=> returns all rows from both tables

```
 SELECT e.ename,d.dname
FROM emp e  FULL OUTER JOIN dept d
```

```
    ON e.deptno = d.deptno ;

        A       ACCOUNTS
        B       RESEARCH
        C       SALES
        D       RESEARCH
        E       NULL            => unmatched from emp
        NULL    OPERATIONS=> unmatched from dept

  Displaying only unmatched records :-
  ============================

  left side table :-
  ============

     SELECT e.ename,d.dname
     FROM emp e  LEFT  OUTER JOIN dept d
     ON e.deptno = d.deptno
     WHERE d.dname IS NULL ;

        E       NULL

 right side table :-
==============

     SELECT e.ename,d.dname
     FROM emp e  RIGHT  OUTER JOIN dept d
     ON e.deptno = d.deptno
     WHERE e.ename IS NULL ;

      NULL              OPERATIONS

  both tables :-
  ==========

    SELECT e.ename,d.dname
     FROM emp e FULL  OUTER JOIN dept d
     ON e.deptno = d.deptno
    WHERE e.ename IS NULL
              OR
              d.dname IS NULL ;

        E       NULL
```

NULL OPERATIONS

Ex 1 :-

emp			projects			
empid	ename sal	projid	projid	pname	duration	cost
100	1000		1000	ABC		
101	1001		1001	KLM		
102	null		1002	DBS		

1 display employee details with project details ?

2 display employee details with project details and also display employees not assigned to any project ?

3 display only the projects where no employee assigned to it ?

Ex 2 :-

T1	T2
F1	F1
1	1
2	2
1	1
2	2
NULL	NULL
NULL	NULL

=> find no of rows return by following operations ?

1 EQUI JOIN => 8
2 LEFT OUTER JOIN => 10
3 RIGHT OUTER JOIN => 10
4 FULL OUTER JOIN => 12

NON-EQUI JOIN :-
==========

=> non equi join is performed between the tables not sharing a common field
=> join condition is not based on "=" and it is based on > < between operators

Ex :-

EMP

empno	ename	sal
100	A	3000
101	B	2000
102	C	5000
103	D	3000
104	E	1500

SALGRADE

grade	losal	hisal
1	800	1000
2	1001	2000
3	2001	3000
4	3001	4000
5	4001	9999

DISPLAY ename sal grade ?

```
SELECT e.ename,e.sal,s.grade
FROM emp e JOIN salgrade s
ON e.sal  BETWEEN s.losal AND s.hisal ;
```

```
A    3000   3
B    2000   2
C    5000   5
D    3000   3
E    1500   2
```

=> display grade 3 employee list ?

```
SELECT e.ename,e.sal,s.grade
FROM emp e JOIN salgrade s
ON e.sal  BETWEEN s.losal AND s.hisal
WHERE s.grade = 3 ;
```

=> display ENAME DNAME GRADE ?

```
SELECT  e.ename,d.dname,s.grade
FROM emp e
INNER JOIN dept d
ON e.deptno = d.deptno
JOIN salgrade s
ON e.sal BETWEEN s.losal AND s.hisal ;
```

(emp join dept) join salgrade

FROM emp e INNER JOIN dept d :-

==========

EMP DEPT

empno	ename	sal	deptno	deptno	dname	loc
1	A	3000	10	10	ACCOUNTS	NEW YORK
2	B	4000	20	20	RESEARCH	
3	C	5000	30	30	SALES	
4	D	3000	20	4O	OPERATIONS	
5	E	2000	10			

ON e.deptno = d.deptno :-

							SALGRADE	
						GRADE	LOSAL	HISAL
1	A	3000	10	10	ACCOUNTS	1	800	1000
2	B	4000	20	20	RESEARCH	2	1001	2000
3	C	5000	30	30	SALES	3	2001	3000
4	D	3000	20	20	RESEARCH	4	3001	4000
5	E	2000	10	10	ACCOUNTS	5	4001	9999

ON e.sal BETWEEN s.losal AND s.hisal :-

1	A	3000	10	10	ACCOUNTS	3
2	B	4000	20	20	RESEARCH	4
3	C	5000	30	30	SALES	5
4	D	3000	20	20	RESEARCH	3
5	E	2000	10	10	ACCOUNTS	2

SELECT e.ename,d.dname,s.grade :-

A	ACCOUNTS	3
B	RESEARCH	4
C	SALES	5
D	RESEARCH	3
E	ACCOUNTS	2

26-oct-26

SELF JOIN :-

=> joining a table to itself is called self join
=> In self join a record in one table joined with another record of same table
=> To perform self join the same table must be declared two times with different alias in FROM

clause

FROM emp x JOIN emp y

emp x			emp y		
empno	ename	mgr	empno	ename	mgr
7369	smith	7902	7369	smith	7902
7499	allen	7698	7499	allen	7698
7566	jones	7839	7566	jones	7839
7698	blake	7839	7698	blake	7839
7839	king	null	7839	king	null
7902	ford	7566	7902	ford	7566

=> display ENAME MGRNAME ?

```
SELECT x.ename,y.ename as manager
FROM emp x JOIN emp y
ON x.mgr = y.empno ;
```

smith	ford
allen	blake
jones	king
blake	king
ford	jones

=> display employee list reporting to blake ?

```
SELECT x.ename,y.ename as manager
FROM emp x JOIN emp y
ON x.mgr = y.empno
WHERE y.ename='blake' ;
```

=> display blake's manager name ?

```
SELECT x.ename,y.ename as manager
FROM emp x JOIN emp y
ON x.mgr = y.empno
WHERE x.ename='blake' ;
```

=> display employees earning more than their manager >

```
SELECT x.ename,x.sal,
       y.ename as manager,y.sal as mgrsal
```

```
    FROM emp x JOIN emp y
    ON x.mgr = y.empno
    WHERE  x.sal > y.sal ;
```

=> employees who are senior to their manager ?

```
     SELECT  x.ename,x.hiredate,
                  y.ename as manager,y.hiredate as mgrhire
    FROM emp x JOIN emp y
    ON x.mgr = y.empno
    WHERE  x.hiredate  <   y.hiredate ;
```

=>

TEAMS

id	country
1	IND
2	AUS
3	ENG

output :-

IND VS AUS
IND VS ENG
AUS VS ENG

TEAMS A		TEAMS B	
ID	COUNTRY	ID	COUNTRY
1	IND	1	IND
2	AUS	2	AUS
3	ENG	3	ENG

```
SELECT  A.COUNTRY||' VS '||B.COUNTRY
FROM  TEAMS A JOIN TEAMS B
ON     A.ID < B.ID
```

IND AUS
IND ENG'
AUS ENG

=> display ENAME DNAME GRADE MNAME ?

```
SELECT e.ename,
```

```
        d.dname,
        s.grade,
        m.ename
FROM emp e
INNER JOIN dept d
ON e.deptno = d.deptno
JOIN salgrade s
ON e.sal BETWEEN s.losal AND s.hisal
JOIN emp m
ON e.mgr = m.empno ;
```

CROSS JOIN :-
==========

=> cross join returns cross product or cartesian product of two tables

A = 1,2
B = 3,4
AXB = (1,3) (1,4) (2,3) (2,4)

=> if cross join performed between two table then all records of 1st table joined with all records of 2nd table.

=> join condition is not required to perform cross join

ex 1 :-

```
SELECT e.ename,d.dname
FROM emp e CROSS JOIN dept d ;
```

ex 2 :-

T1			T2		
id	name	price	id	name	price
1	PIZZA	100	1	juice	60
2	BURGER	80	2	thumsup	40

output :-

PIZZA + juice	160
PIZZA + thumsup	140
BURGER + juice	140
BURGER + thumsup	120

```
SELECT  T1.NAME|| ' + '||T2.NAME  AS   NAME,T1.PRICE + T2.PRICE AS TOTAL
 FROM T1 CROSS JOIN T2
```

==========

SET OPERATORS :-

==========

UNION
UNION ALL
INTERSECT
MINUS

A = 1,2,3,4
B = 1,2,5,6

A UNION B => 1,2,3,4,5,6
A UNION ALL B => 1,2,3,4,1,2,5,6
A INTERSECT B => 1,2
A MINUS B => 3,4
B MINUS A => 5,6

=> In oracle set operation performed between the rows return by two queries

```
SELECT  statement 1
UNION / UNION ALL / INTERSECT / MINUS
SELECT statement 2 ;
```

Rules :-

1 no of columns return by both queries must be same
2 corresponding columns datatype must be same

Query 1 :-

========

```
SELECT job FROM emp WHERE deptno = 20 ;
```

CLERK
MANAGER
ANALYST
CLERK

ANALYST

Query 2 :-
========

```
SELECT job FROM emp WHERE deptno = 30 ;
```

SALESMAN
SALESMAN
SALESMAN
MANAGER
SALESMAN
CLERK

UNION :-
=======

=> combines rows return by two queries
=> eliminates duplicates

```
SELECT job FROM emp WHERE deptno = 20
union
SELECT job FROM emp WHERE deptno = 30 ;
```

CLERK
MANAGER
ANALYST
SALESMAN

```
SELECT job,sal FROM emp WHERE deptno = 20
union
SELECT job,sal FROM emp WHERE deptno = 30 ;
```

CLERK	800
MANAGER	2975
ANALYST	3000
CLERK	1100
SALESMAN	1600
SALESMAN	1250
MANAGER	2850
SALESMAN	1500
CLERK	950

=> difference between UNION & JOIN ?

	union	join
1	combines rows	combines columns
2	horizontal merge	vertical merge
3	performed between two queries	performed between two tables
4	performed between two similar structures	performed between two dissimilar structures

=>

T1	T2
F1	F1
10	40
20	50
30	60

T1 UNION T2 :-

10
20
30
40
50
60

T1 JOIN T2 :-

10	40
20	50
30	60

=>

EMP_US

eno	ename	dno
100	A	10
101	B	20

EMP_IND

eno	ename	dno
200	K	10
201	S	20

DEPT

dno	dname	loc
10	HR	
20	IT	

=> display total employees list ?

```
SELECT * FROM EMP_US
UNION
SELECT * FROM EMP_IND ;
```

=> display employees working at US loc with dept details ?

```
SELECT  e.*,d.*
FROM  emp_us e  INNER JOIN dept d
ON e.dno = d.dno ;
```

```
100  A        10       10       HR
101  B        20       20       IT
```

=> display total employees with dept details ?

```
SELECT  e.*,d.*
FROM  emp_us e  INNER JOIN dept d
ON e.dno = d.dno
UNION
SELECT  e.*,d.*
FROM  emp_ind e  INNER JOIN dept d
ON e.dno = d.dno ;
```

UNION ALL :-
==========

=> combines rows return by two queries
=> duplicates are not eliminated

```
SELECT job FROM emp WHERE deptno = 20
union all
SELECT job FROM emp WHERE deptno = 30 ;
```

```
CLERK
MANAGER
ANALYST
CLERK
ANALYST
SALESMAN
 SALESMAN
```

SALESMAN
MANAGER
SALESMAN
CLERK

=> difference between UNION & UNION ALL ?

	union	union all
1	eliminates duplicates	doesn't eliminate duplicates
2	slower	faster

INTERSECT :-
==========

=> returns common values from the output of two select stmts

```
SELECT job FROM emp WHERE deptno = 20
INTERSECT
SELECT job FROM emp WHERE deptno = 30 ;
```

CLERK
MANAGER

MINUS :-
=========

=> returns values present in 1st query output and not present in 2nd query output

```
SELECT job FROM emp WHERE deptno = 20
MINUS
SELECT job FROM emp WHERE deptno = 30 ;
```

ANALYST

```
SELECT job FROM emp WHERE deptno = 30
MINUS
SELECT job FROM emp WHERE deptno = 20 ;
```

SALESMAN

=>

T1	T2
F1	C1
1	1
2	2
3	3
10	40
20	50
30	60

=> write the output for the following operations ?

1 EQUI JOIN
2 LEFT OUTER JOIN
3 RIGHT OUTER JOIN
4 FULL OUTERR JOIN
5 UNION
6 UNION ALL
7 INTERSECT
8 MINUS

==========
=

SUB-QUERIES / NESTED QUERIES :-
==========

=> a query in anoither query sub-query or nested query
=> one query is called inner / child / sub -query
=> other query is called outer / parent / main query
=> first oracle executes inner query then it executes outer query
=> output of inner query is input to outer query
=> we can write sub-queries in

1 WHERE clause / HAVING clause
2 FROM clause
3 SELECT clause

sub-queries in WHERE clause :-
==========

```
SELECT columns
FROM tabname
```

WHERE colname OP (SELECT statement);

=> we write sub-queries in where clause when value is unknown

Types of sub-queries :-
---

1 single row sub-queries
2 multi row sub-queries
3 co-related sub-queries

single row sub-queries :-
==========

=> if sub-query returns one value then it is called single row sub-query
=> OP must be any relational operator like = > >= < <= <>

ex :-

=> display employees earning more than blake ?

```
SQL>SELECT *
      FROM emp
      WHERE sal  >   ( SELECT sal FROM emp WHERE ename='blake' )  ;
                    ----------------------------------------------------------------------
                          2850
```

=> employees who are senior to king ?

```
SQL>SELECT *
      FROM emp
      WHERE hiredate <  (SELECT hiredate FROM emp WHERE ename='king') ;
                    ----------------------------------------------------------------------
                          17-NOV-81
```

=> display name of the employee earning max salary ?

```
SQL>SELECT ename
      FROM emp
      WHERE sal = MAX(sal) ;   => ERROR

SQL>SELECT ename
      FROM emp
```

```
        WHERE sal =  (SELECT MAX(sal) FROM emp) ;
                    ------------------------------------------
                          5000
```

=> employee name having max experience ?

```
 SQL>SELECT ename
       FROM emp
       WHERE hiredate = (SELECT MIN(hiredate) FROM emp);
                        ------------------------------------------------
                              17-DEC-80
```

=> display 2nd max salary ?

```
  SELECT  MAX(sal)
  FROM emp
  WHERE sal < (SELECT MAX(sal) FROM emp) ;
                    ------------------------------------------
                              5000
```

using FETCH clause :-

==========

```
   SELECT distinct sal
   FROM emp
   ORDER BY sal DESC
   OFFSET 1 ROW FETCH NEXT 1 ROW ONLY ;
```

=> name of the employee earning 2nd max salary ?

```
SQL>SELECT ename
       FROM emp
       WHERE sal =  ( SELECT  MAX(sal)
                      FROM emp
                      WHERE sal < (SELECT MAX(sal) FROM emp));
                                   ------------------------------------------
                                  5000
                  ------------------------------------------------------------
                       3000
```

=> name of the employees working at NEW YORK loc ?

using sub-query :-

```
SQL>SELECT ename
        FROM emp
        WHERE deptno =  ( SELECT deptno FROM dept WHERE loc='NEW YORK');
                        ------------------------------------------------------------------------------
                        10
```

using join :-

```
SQL>SELECT e.ename
        FROM emp e INNER JOIN dept d
        ON e.deptno = d.deptno
        WHERE d.loc = 'NEW YORK' ;
```

=> display ENAME   DNAME working at NEW YORK loc ?

using join :-

```
SQL>SELECT e.ename,d.dname
        FROM emp e INNER JOIN dept d
        ON e.deptno = d.deptno
        WHERE d.loc = 'NEW YORK' ;
```

using sub-query :- NOT POSSIBLE

---

difference between join & sub-query ?

1 to display data from one table and condition based on different table then we can use sub-query or join

2 to display data from two tables then use join operation

28-OCT-26

=> delete the employee having max experience ?

```
SQL>DELETE
        FROM emp
        WHERE hiredate = (SELECT MIN(hiredate) FROM emp) ;
```

=> hike the salary of the employee by 10% having max experience ?

```
SQL>UPDATE emp
      SET sal = sal + (sal*0.1)
       WHERE hiredate = (SELECT MIN(hiredate) FROM emp) ;
```

=> swap employee salaries whose empno = 7369,7499 ?

before swap		after swap	
7369	800	7369	1600
7499	1600	7499	800

```
SQL>UPDATE emp
      SET sal =  CASE empno
                 WHEN 7369 THEN  (SELECT sal FROM emp WHERE empno=7499)
                 WHEN 7499 THEN  (SELECT sal FROM emp WHERE empno=7369)
                 END
      WHERE empno IN (7369,7499) ;

SQL> UPDATE emp
      SET sal =  CASE empno
                 WHEN 7369 THEN 1600
                  WHEN 7499 THEN 800
                 END
      WHERE empno IN (7369,7499) ;
```

multi-row sub-queries :-
==========

=> if sub-query returns more than one value then it is called multi-row sub-query

```
SELECT columns
FROM tabname
WHERE  colname  OP  (SELECT statement) ;
```

=> OP must be IN,NOT IN,ANY,ALL

single	multi
=	IN
<>	NOT IN
>	>ANY >ALL
<	<ANY <ALL

Ex :-

=> employees working at NEW YORK,CHICAGO locations ?

```
SQL>SELECT *
        FROM emp
        WHERE deptno IN  (SELECT deptno
                          FROM dept
                          WHERE loc IN ('NEW YORK','CHICAGO'))
```

ANY & ALL operators :-
==========

=> use ANY & ALL operators for >   <   comparison with multiple values

WHERE sal > ANY(1000,2000,3000)

if sal=800 false
        1500 true
    4000 true

WHERE sal < ANY(1000,2000,3000)

if sal = 800 true
        1500 true
    4000 false

WHERE sal > ALL(1000,2000,3000)

if sal=800 false
    1500 false
    4000 true

WHERE sal < ALL(1000,2000,3000)

if sal=800 true
    1500 false
    4000 false

=> display employees earning more than all managers ?

```
SQL>SELECT *
        FROM emp
        WHERE sal >  ALL (SELECT sal FROM emp WHERE job='MANAGER') ;
                          ---------------------------------------------------------------------
                          2975
                          2850
                          2450
```

=> display employees earning more than atleast one manager ?

```
SQL> SELECT *
        FROM emp
        WHERE sal >  ANY (SELECT sal FROM emp WHERE job='MANAGER') ;
```

co-related sub-queries :-
==========

=> if inner query references values of outer query then it is called co-related sub-query.

=> execution starts from outer query and inner query is executed no of times depends on no of rows return by outer query.

=> use co-related sub-query to execute sub-query for each row return by outer query

Ex 1 :-

EMP

empno	ename	sal	deptno
1	A	6000	10
2	B	3000	20
3	C	5000	30
4	D	4000	20
5	E	3000	10

=> employees earning more than avg sal ?

```
SQL>   SELECT *
       FROM emp
        WHERE sal >  (SELECT AVG(sal) FROM emp) ;
                      ----------------------------------------
                      4200
```

=> employees earning more than avg sal of their dept ?

```
SQL>SELECT empno,ename,sal,deptno
       FROM emp  x
       WHERE sal >  (SELECT AVG(sal) FROM emp WHERE deptno = x.deptno ) ;
```

empno	ename	sal	deptno			
1	A	6000	10	6000 > (where deptno = 10)	4500	TRUE
2	B	3000	20	3000 > (where deptno = 20)	3500	FALSE
3	C	5000	30	5000 > (where deptno = 30)	5000	FALSE
4	D	4000	20	4000 > (where deptno = 20)	3500	TRUE
5	E	3000	10	3000 > (where deptno=10)	4500	FALSE

=> employees earning max salary in their dept ?

```
SQL>SELECT empno,ename,sal,deptno
      FROM emp  x
      WHERE sal =  (SELECT MAX(sal) FROM emp WHERE deptno = x.deptno ) ;
```

empno	ename	sal	deptno	
1	A	6000	10	6000 = (6000) TRUE
2	B	3000	20	3000 = (4000) FALSE
3	C	5000	30	5000= (5000) TRUE
4	D	4000	20	4000 = (4000) TRUE
5	E	3000	10	3000 = (6000) FALSE

=> display top 3 max salaries ?

```
SQL>SELECT DISTINCT a.sal
     FROM emp a
     WHERE  3 > (SELECT COUNT(DISTINCT b.sal)
                 FROM emp b
                 WHERE a.sal < b.sal)
     ORDER BY a.sal DESC ;
```

EMP a	EMP b		
sal	sal		
5000	5000	3 > (0)	TRUE
1000	1000	3 > (4)	FALSE
3000	3000	3 > (1)	TRUE
2000	2000	3 > (3)	FALSE
2975	2975	3 > (2)	TRUE

29-oct-26

=> display 3rd max salary ?

```
SQL> SELECT DISTINCT a.sal
     FROM emp a
     WHERE (3-1) =  (SELECT COUNT(DISTINCT b.sal)
                     FROM emp b
                     WHERE a.sal < b.sal)
     ORDER BY a.sal DESC ;
```

=> display Nth max salary ?

```
SQL>   SELECT DISTINCT a.sal
       FROM emp a
       WHERE (&N-1) =  (SELECT COUNT(DISTINCT b.sal)
                        FROM emp b
                        WHERE a.sal < b.sal)
       ORDER BY a.sal DESC ;
```

ROWID :-
========

=> returns physical address of a row i.e. where exactly record stored in memory
=> rowid is called psuedo column (false) because it is not a column but acts like a column
=> rowids are used to delete duplicate records

psuedo column :-
==========

1 SYSDATE
2 SYSTIMESTAMP
3 USER
4 LEVEL
5 ROWID
6 ROWNUM

=> display rowids of employees ?

```
SQL>SELECT rowid,empno,ename,sal FROM emp ;
```

Deleting duplicate records :-
==========

EMP44

eno	ename	sal		rowid
1	A	5000		AAA
2	B	6000		AAB
3	C	7000		AAC
1	A	5000	=> duplicate	AAD
2	B	6000	=> duplicate	AAE

```
SQL>DELETE FROM emp44 WHERE rowid LIKE '%AAD' ;
SQL>DELETE FROM emp44 WHERE rowid LIKE '%AAE' ;
```

=> deleting duplicates row-by-row is difficult if table contains huge number of
duplicate rows , so execute the following query to delete all duplicate rows in a table

method 1 :- (co-related sub-query)

```
SQL>DELETE
        FROM emp44 a
        WHERE rowid <>  (SELECT  MIN(rowid)
                         FROM emp44
                         WHERE eno = a.eno
                                  AND
                                  ename = a.ename
                                  AND
                                  sal = a.sal
                          ) ;
```

EMP44

eno	ename	sal	rowid			
1	A	5000	AAA	<>	(AAA)	FALSE
2	B	6000	AAB	<>	(AAB)	FALSE
3	C	7000	AAC	<>	(AAC)	FALSE
1	A	5000	AAD	<>	(AAA)	TRUE
2	B	6000	AAE	<>	(AAB)	TRUE

method 2 :- (multi row sub-query)

==========

```
SQL>DELETE
       FROM emp44
       WHERE rowid  NOT  IN (
                             SELECT MIN(rowid)
                             FROM emp44
                             GROUP BY eno,ename,sal
                            ) ;
```

AAA
AAB
AAC

INLINE views :-

============

=> sub-queries in FROM clause are called inline views

```
SELECT columns
FROM (SELECT statement)   <name>
WHERE cond;
```

=> sub-query output acts like a table for outer query.
=> inline views are used in following scenarios

1 to control the order of execution of clauses
2 to use result of one opertation in another operation
3 to join two query outputs

controlling order of execution :-
==========

default order :-
==========

```
FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
```

```
SELECT                                                    SELECT
FROM  ========================================>      FROM (SELECT
WHERE                                                FROM
ORDER BY                                           ORDER BY )
                                                  WHERE
```

Ex 1 :-

=> find the ranks of the employees based on sal and highest paid should get 1st rank ?

```
SQL>SELECT empno,ename,sal,
                DENSE_RANK() OVER ( ORDER BY sal DESC)  as rnk
      FROM emp ;
```

=> above query displays ranks of all employees but to display top 5 employees

```
SQL>SELECT empno,ename,sal,
                DENSE_RANK() OVER ( ORDER BY sal DESC)  as rnk
```

```
        FROM emp
       WHERE  rnk <= 5 ;  => ERROR
```

=> column alias cannot be used in where clause because where clause is executed before select , to overcome this problem use INLINE views

```
SQL>SELECT *
       FROM (SELECT empno,ename,sal,
                    DENSE_RANK() OVER ( ORDER BY sal DESC)  as rnk
             FROM emp)  E
       WHERE  rnk <= 5 ;
```

=> display top 3 max salaries ?

```
SQL> SELECT DISTINCT sal
       FROM (SELECT  sal,
                    DENSE_RANK() OVER ( ORDER BY sal DESC)  as rnk
             FROM emp)  E
       WHERE  rnk <= 3  ;
```

30-oct-26

ROWNUM :-
==========

=> rownum returns record number
=> rownum is also a psuedo column
=> rownum is useful when fetching records from table based on record number

Ex :-

```
SQL>SELECT rownum,empno,ename,sal FROM emp ;

        9       7839    king    5000

SQL>SELECT rownum,empno,ename,sal FROM emp WHERE sal >= 2000;

        5       7839    king    5000
```

=> rownum is not based on table and it is based on query output , if query output changes rownum also changes.

=> display first 5 rows from emp table ?

```
SQL>SELECT  empno,ename,sal
        FROM emp
        WHERE rownum <= 5 ;
```

=> display 5th row ?

```
SQL>  SELECT  empno,ename,sal
        FROM emp
        WHERE rownum = 5 ;    =>  NO ROWS
```

NOTE :-

=> In where condition with rownum = > >= will not work
and only < <= operators will work , to overcome this
use INLINE views.

```
SQL>SELECT *
        FROM  (SELECT rownum as rno,empno,ename,sal FROM emp)  E
        WHERE rno = 5 ;

        WHERE rno IN (5,10,15);

        WHERE rno BETWEEN 5  AND 10 ;

        WHERE MOD(rno,2) = 0 ;
```

=> display last 3 rows from emp table ?

```
SQL> SELECT *
        FROM  (SELECT rownum as rno,empno,ename,sal FROM emp)  E
        WHERE rno  >= (SELECT count(*)-2   FROM emp) ;

SQL>SELECT *
        FROM  (SELECT rownum as rno,empno,ename,sal FROM emp)  E
        WHERE rno  <= 3
                OR
                rno >= 12 ;
```

CTE :-
======

=> CTE stands for common table expression.

=> using CTE we can give name to the query output and we can refer that name in another query.

=> CTE simplifies complex scenario

=> CTE improves query readability

```
WITH  <CTE1>
AS
   (SELECT statement) ,
<CTE2>
AS
   (SELECT statement) ,
----------------
```

SELECT statement

ex 1 :- display top 5 highest paid employees ?

```
WITH E
 AS
   (SELECT empno,ename,sal,
              dense_rank() over (order by sal desc) as rnk
    FROM emp)
SELECT * FROM E WHERE rnk <= 5 ;
```

ex 2 :- joining two tables without common field

T1	T2
F1	C1
10	A
20	B
30	C

output :-

10	A
20	B
30	C

Query 1 :-

```
SELECT ROWNUM  as rno,f1 FROM T1 ;

1       10
2       20
3       30
```

Query 2 :-

```
SELECT ROWNUM as rno,c1 FROM T2 ;

1       A
2       B
3       C

WITH  A
 AS
    (SELECT ROWNUM  as rno,f1 FROM T1) ,
B
AS
   ( SELECT ROWNUM as rno,c1 FROM T2)
 SELECT A.f1 , B.c1
 FROM  A  INNER JOIN B
ON A.rno = B.rno ;
```

ex 3 :- seperate positives and negatives ?

```
 T1
 AMT
1000
-300
5000
-500
3000
-800

output :-          POS          NEG
                1000          -300
                5000          -500
                3000          -800

WITH P
AS
    ( SELECT rownum as rno ,amt  FROM T1 WHERE amt > 0) ,
```

```
 N
AS
   ( SELECT rownum as rno,amt FROM T1 WHERE amt < 0 )
 SELECT P.amt , N.amt
 FROM P INNER JOIN N
 ON  P.rno = N.rno ;
```

==========

```
 PIVOT operator :-
 ==============

 => operator used to convert rows into columns
 => using PIVOT we can display data in matrix form

 SELECT  *
 FROM  (SELECT required data)  <name>
 PIVOT
 (
   aggr-expr  FOR  colname IN (v1,v2,v3,---)
  )
  ORDER BY colnam ASC/DESC ;
```

Ex 1 :-

```
 SELECT *
 FROM (SELECT deptno,job,sal FROM emp)  E
 PIVOT
 (
   SUM(sal)  FOR deptno IN (10,20,30)
 )
ORDER BY job ASC ;
```

JOB	10	20	30	
ANALYST		6000		
CLERK	1300	1900	950	
MANAGER	2450	2975	2850	
PRESIDENT	5000			
SALESMAN			5600	

Ex 2 :-

	1	2	3	4
1980	?	?	?	?
1981	?	?	?	?
1982				
1983				

```
SELECT *
FROM  (SELECT  TO_CHAR(hiredate,'yyyy') as year,
               TO_CHAR(hiredate,'q') as qrt,
               empno
       FROM emp)  E
PIVOT
(
   COUNT(empno) FOR qrt IN (1,2,3,4)
)
ORDER BY year ASC ;
```

Ex 3 :-

STUDENT

sno	sname	subject	marks
1	A	mat	80
1	A	phy	60
1	A	che	70
2	B	mat	60
2	B	phy	70
2	B	che	50

output :-

sno	sname	mat	phy	che
1	A	80	60	70
2	B	60	70	50

==========

=

31-oct-26

creting new table from existing table (creating repica) :-
==========

```
CREATE TABLE <new-tabname>
AS
SELECT columns  FROM  <old-tabname> [WHERE cond] ;
```

ex 1 :- copying complete table

```
CREATE TABLE emp33
AS
SELECT * FROM emp ;
```

ex 2 :- copy only specific rows and cols

```
CREATE TABLE emp44
AS
SELECT empno,ename,job,sal FROM emp  WHERE job IN ('CLERK','MANAGER') ;
```

ex 3 :- copy only structure (columns) but not data (rows)

```
CREATE TABLE emp55
AS
SELECT * FROM emp WHERE  1=2 ;
```

copying data from one table to another table :-
==========

```
INSERT INTO <target-table>
SELECT columns FROM <source-table> [WHERE cond] ;
```

ex :- copy data from emp to emp55 ?

```
    INSERT INTO emp55
    SELECT * FROM emp ;
```

MERGE command :-
==========

=> merge is the combination of insert and update
=> command used to merge data into a table
=> using merge command we can apply changes made to one table to another table
=> command used to replicate changes

```
MERGE INTO <target-table>   <alias>
USING <source-table>  <alias>
ON (condition)
WHEN MATCHED THEN
        update
WHEN NOT MATCHED THEN
        insert ;
```

ex :-

step 1 :- create source table

```
CREATE TABLE CUSTS
(
 cid  NUMBER(2),
 cname VARCHAR2(10),
 city  VARCHAR2(10)
 );

INSERT INTO CUSTS VALUES(1,'A','HYD'),(2,'B','BLR')
```

step 2 :- create replica for custs

```
CREATE TABLE CUSTT
AS
SELECT * FROM CUSTS ;
```

step 3 :- modify source table

```
INSERT INTO CUSTS VALUES(3,'C','DEL');

UPDATE CUSTS SET CITY='MUM' WHERE CID=2;
```

CUSTS			CUSTT		
cid	cname	city	cid	cname	city
10	A	HYD	10	A	HYD
11	B	MUM=>U	11	B	BLR
12	C	DEL => I			

step 4 :- apply the changes to custt using merge command

```
MERGE INTO CUSTT   T
USING CUSTS S
ON (S.CID = T.CID)
WHEN MATCHED THEN
   UPDATE SET T.CITY = S.CITY
WHEN NOT MATCHED THEN
    INSERT VALUES(S.CID,S.CNAME,S.CITY);
```

CUSTS

cid	cname	city
10	A	HYD
11	B	MUM=>U
12	C	DEL => I

CUSTT

cid	cname	city
10	A	HYD
11	B	MUM
12	C	DEL

==========

DATABASE TRANSACTIONS :-

==========

=> a transaction is a unit of work that contains one or more dmls and must be saved as a whole or must be cancelled as a whole.

ex :- acct1----------1000----------acct2

update1 (bal = bal - 1000) | update2 (bal = bal + 1000)

successful	failed	invalid
failed	successful	invalid
successful	successful	valid
failed	failed	valid

=> every transaction must gurantee a property called atomocity i.e. all or none , if transaction contains multiple operations if all are successful then it must be saved , if one of the operation fails then entire txn must be cancelled.

=> the following commands provided by oracle to handle transactions called TCL commands

1 COMMIT => to save txn
2 ROLLBACK => to cancel txn
3 SAVEPOINT => to cancel pat of the txn

=> every txn has a begin point and an end point
=> a txn begins implicitly whenever user submits DML commands to oracle
=> a txn ends with any of the following commands

1 COMMIT / ROLLBACK
2 DDL command (txn ends with commit)

ex 1 :-

```
CREATE TABLE a(a  NUMBER(2));  COMMIT
INSERT INTO a VALUES(10);          TXN BEGINS T1
INSERT INTO a VALUES(20);
INSERT INTO a VALUES(30);
INSERT INTO a VALUES(40);
COMMIT ;                                TXN ENDS
```

=> if txn ends with COMMIT then it is called successful txn and operations are saved

```
CREATE TABLE a(a  NUMBER(2));
INSERT INTO a VALUES(10);
INSERT INTO a VALUES(20);
INSERT INTO a VALUES(30);
INSERT INTO a VALUES(40);
COMMIT ;
```

ex 2 :-

```
 CREATE TABLE a(a  NUMBER(2));    COMMIT
INSERT INTO a VALUES(10);          TXN BEGINS T1
INSERT INTO a VALUES(20);
INSERT INTO a VALUES(30);
INSERT INTO a VALUES(40);
ROLLBACK;                               TXN ENDS
```

=> if txn ends with ROLLBACK then it is called aborted txn and opertions are cancelled

Ex 3 :-

```
CREATE TABLE a(a  NUMBER(2));   COMMIT
INSERT INTO a VALUES(10);          TXN BEGINS T1
INSERT INTO a VALUES(20);
CREATE TABLE b(b NUMBER(2));   TXN ENDS WITH COMMIT
INSERT INTO a VALUES(30);          TXN BEGINS T2
INSERT INTO a VALUES(40);
ROLLBACK;                                TXN ENDS WITH ROLLBACK
```

SAVEPOINT :-
==========

=> we can declare savepoint and we can rollback up to the savepoint
=> using savepoint we can cancel part of the txn

```
CREATE TABLE a(a  NUMBER(2));
INSERT INTO a VALUES(10);
INSERT INTO a VALUES(20);
SAVEPOINT SP1;
INSERT INTO a VALUES(30);
INSERT INTO a VALUES(40);
SAVEPOINT SP2;
INSERT INTO a VALUES(50);
INSERT INTO a VALUES(60);
ROLLBACK TO SP1 ;

  SELECT * FROM A ;

10
20
```

2-nov-26

Locking :-
======

=> accessing same data by no of users at the same time is called concurrent access
=> when data acceessed concurrently , users encounters following problems

1. dirty read
2. lost update
3. phantom read
4. non repeatable read

=> to overcome the above problems every db system supports a mechanism called locking

=> locks are 2 types / modes

1 shared lock (S)
2 exclusive lock (X)

=> shared lock is applied whenever user try to read data (select stmt).
=> exclusive lock is applied whenever user try to modify data (dml command)

	S	X
S	yes	yes
X	yes	no

ex :-

system	batch60
1 UPDATE BATCH60.EMP SET SAL=2000 WHERE EMPNO = 7369 ;	2 UPDATE EMP SET SAL = 3000 WHERE EMPNO = 7369 ;
txn begins locked updated	----------wait----------
3 COMMIT; (lock releases)	4 1 row updated

DeadLock :-
=========

=> deadlock is the situation where two users mutually waits for one another , when deadlock occurs oracle throws error so that one transaciton can be cancelled and continue another transaction.

ex :-

system	batch60

1 UPDATE BATCH60.EMP
SET SAL=2000
WHERE EMPNO = 7369 ;

locked + updated

2 UPDATE EMP
SET SAL = 3000
WHERE EMPNO = 7499 ;

locked + updated

3
UPDATE BATCH60.EMP
SET SAL=2000
WHERE EMPNO = 7499;

----------wait----------

4
UPDATE EMP
SET SAL = 3000
WHERE EMPNO = 7369;

----------wait----------

----------deadlock----------

5 ERROR

6 ROLLBACK ;

7 COMMIT;

what are ACID properties / Transaction properties ?

=> every txn in db must gurantee 4 properties called ACID properties

A atomocity
C consistency
I isolation
D durability

Atomocity :-
==========

=> a transaction is treated as a single unit of work. Either all operations succeed and or none , leaving the database unchanged if a failure occurs

consistency :-
==========

=> : A transaction must transform the database from one valid state to another valid state,

isolation :-
========

=> one transaction should not interfere into another transaction

durabiilty :-
=======

=> Once a transaction is successfully completed and committed, its changes are permanently saved in the database. The data will survive even if a sudden power failure or system crash happens immediately afterward

==========
==

DB Security :-
==========

1 USERS => provides security at db level
2 PRIVILEGES => provides security at table level
3 VIEWS => provides security at row & col level

server
database (users)
tables (privileges)
rows & cols (views)

creating user in db :-
==========

SYSTEM :-
========

SQL>CREATE USER NARESH IDENTIFIED BY ORACLE
DEFAULT TABLESPACE USERS
QUOTA UNLIMITED ON USERS;

SQL>GRANT CONNECT,RESOURCE TO NARESH ;

PRIVILEGES :-
==========

=> privileges means permissions
=> permissions granted on tables to users using grant command

GRANT <privileges> ON <tabname> TO <usernames>;

ex :-

BATCH60 :-
==========

SQL>GRANT ALL ON EMP TO NARESH ;

NARESH :-
=========

SQL>SELECT * FROM BATCH60.EMP;
SQL>UPDATE BATCH60.EMP SET SAL = 3000 WHERE EMPNO = 7369 ;
SQL>DELETE FROM BATCH6O.EMP WHERE EMPNO = 7369 ;

NOTE :- changes made by NARESH visible to BATCH60 after commit.

Granting specific privileges :-
=======================

BATCH60 :-
==========

SQL>GRANT SELECT,UPDATE ON DEPT TO NARESH ;

NARESH :-
=========

SQL>SELECT * FROM BATCH60.DEPT;
SQL>UPDATE BATCH60.DEPT   SET LOC='HYD' WHERE DEPTNO = 10;
SQL>DELETE FROM BATCH60.DEPT WHERE DEPTNO = 10 ; => ERROR

3-NOV-26

REVOKE command :-
=================

=> command used to take back permissions from user

REVOKE <privileges> ON <tabname> FROM <usernames> ;

Ex :-

BATCH60 :-
=========

SQL>REVOKE ALL ON EMP FROM NARESH ;

NARESH :-
=========

SQL>SELECT * FROM BATCH60.EMP ; => ERROR

DB Objects :-
============

1 TABLES
2 VIEWS
3 SYNONYMS
4 SEQUENCES
5 INDEXES

VIEWS :-
=======

=> a view is a subset of a table i.e. part of the table
=> a view is a virtual table because it doesn't store data and doesn't occupy memory
=> a view always derives data from base table
=> a view is a representation of a query

purpose :-
========

1 to provide security
2 to reduce complexity

Types of views :-
============

1 simple views
2 complex views

simple views :-
============

=> a view said to be simple view if based on single table

```
CREATE VIEW <name>
AS
SELECT statement ;
```

Ex :-

```
SQL>CREATE VIEW V1
        AS
        SELECT empno,ename,job,hiredate,deptno FROM emp ;
```

=> oracle creates view V1 and stores query but not query output (data).

```
SQL>SELECT * FROM V1 ;
```

=> when above query send to oracle , it executes the query as follows

```
SQL>SELECT * FROM ( SELECT empno,ename,job,hiredate,deptno FROM emp);
```

Granting permissions on view to user :-
==========

BATCH60 :-
=========

```
SQL>GRANT ALL ON V1 TO NARESH ;
```

NARESH :-
========

```
SQL>SELECT  * FROM BATCH60.V1 ;
SQL>UPDATE BATCH60.V1 SET JOB='MANAGER' WHERE EMPNO = 7369 ;
```

ROW LEVEL SECURITY :-
==========

BATCH60 :-
=========

```
SQL>CREATE VIEW V2
        AS
        SELECT empno,ename,job,deptno FROM emp WHERE deptno = 20 ;

SQL>GRANT  ALL ON V2 TO NARESH ;
```

NARESH :-
========

```
SQL>SELECT * FROM BATCH60.V2 ;

SQL>INSERT INTO BATCH60.V2 VALUES(8888,'sachin','CLERK',30); => 1 row created
```

=> above insert command executed successfully even though it is violating where cond

WITH CHECK OPTION :-
==========

=> if view created with "WITH CHECK OPTION" then any dml command through view violates where cond that dml is not accepted.

```
SQL>CREATE VIEW V3
        AS
        SELECT empno,ename,job,deptno
        FROM emp
        WHERE deptno = 20
        WITH CHECK OPTION ;

SQL>GRANT ALL ON V3 TO NARESH ;
```

NARESH :-
=========

```
SQL>INSERT INTO BATCH60.V3 VALUES(9999,'kkk','clerk',30);  => ERROR
```

Complex views :-
==========

=> a view said to be complex view

1 if based on multiple tables (joins)
2 if query performs group by
aggregate

distinct
set operations
sub-queries

=> with the help of views a complex query can be converted into simple query

ex 1 :-

```
SQL>CREATE VIEW CV1
        AS
        SELECT e.empno,e.ename,e.sal,
                    d.deptno,d.dname,d.loc
        FROM emp e INNER JOIN dept d
        ON e.deptno = d.deptno ;
```

after creating view, whenever user wants data from emp & dept tables then instead of writing complex join query write the simple query

```
SQL>SELECT * FROM CV1;
```

ex 2 :-

```
SQL>CREATE VIEW CV2
        AS
       SELECT job,MIN(sal) as minsal,
                        MAX(sal) as maxsal,
                        SUM(sal) as totsal,
                        AVG(sal) as avgsal,
                        COUNT(*) as cnt
        FROM emp
        GROUP BY job ;
```

whenever user wants job wise summary then execute the following simple query

```
SQL>SELECT * FROM CV2;
```

=> difference between simple and complex views ?

	simple	complex
1	based on single table	based on multiple tables
2	query performs simple opertions	query performs complex operations

like joins,group by etc

3 updatable (allows dmls) not updatable (doesn't allow dmls)

user_views :-
==========

=> it is a system table that stores views created by user

list of views created by user ?

SQL>SELECT VIEW_NAME FROM USER_VIEWS ;

=> To see the query declared wtih V1 ?

SQL> SELECT TEXT FROM USER_VIEWS WHERE VIEW_NAME = 'V1' ;

Droping :-
=========

SQL>DROP VIEW V1;

==========
===

4-nov-26

synonyms :-
=========

=> a synonym is another name or alternative name for a table or view

=> if table name is lengthy then we can give a simple and short name to the table called synonym and after creating synonym instead of using table name we can use synonym name in SELECT / INSERT / UPDATE / DELETE queries

CREATE SYNONYM <name> FOR <tabname>

ex :- CREATE SYNONYM E FOR EMP ;

=> after creating synonym , use synonym name instead of table name

SQL>SELECT * FROM e ;
SQL>UPDATE e SET comm = 500 WHERE empno = 7369 ;

=> difference between synonym and table alias ?

	SYNONYM	ALIAS
1	permanent	not permanent
2	stored in db	not stored in db
3	scope of the synonym is up to the schema (user)	scope of the alias is up to the query

USER_SYNONYMS :-
==========

=> stores synonyms created by user

SQL>SELECT SYNONYM_NAME,TABLE_NAME FROM USER_SYNONYMS ;

E EMP

Droping synonym :-
==========

SQL>DROP SYNONYM E ;

if we drop table what about synonyms created on table ?

ans :- synonyms are not dropped but cannot be queried

==========

SEQUENCES :-
==========

=> a sequence is also a db object created to generate sequence numbers
and mostly used for primary key columns

=> used to auto increment column values

```
CREATE SEQUENCE <name>
[START WITH <value>]
[INCREMENT BY <value>]
[MAXVALUE <value>]
[MINVALUE <value>]
[CYCLE]
[CACHE <size>] ;
```

ex 1 :-

```
SQL>CREATE SEQUENCE S1
        START WITH 1
        INCREMENT BY 1
        MAXVALUE 5 ;
```

using sequence :-

==========

```
SQL>CREATE TABLE student(sid  NUMBER(2),sname VARCHAR2(10))
```

use sequence S1 to generate sid for every new student ?

```
SQL>INSERT INTO student VALUES(s1.nextval , 'A') ;
SQL>INSERT INTO student VALUES(s1.nextval , 'B') ;
SQL>INSERT INTO student VALUES(s1.nextval , 'C');
SQL>INSERT INTO student VALUES(s1.nextval , 'D');
SQL>INSERT INTO student VALUES(s1.nextval , 'E') ;
SQL>INSERT INTO student VALUES(s1.nextval , 'F');   => ERROR

SQL>SELECT * FROM student ;

        SID     SNAME
        ----------          ----------
        1       A
        2       B
        3       C
        4       D
        5       E
```

ex 2 :-

```
SQL>CREATE SEQUENCE S2
        START WITH 100
```

```
        INCREMENT BY 1
        MAXVALUE 9999;
```

=> use sequence s2 to generate empno for existing employees ?

```
SQL>UPDATE emp SET empno =  s2.nextval ;
```

ex 3 :-

```
BILL
billno          bdate   amt
DM/1126/1
DM/1126/2

CREATE TABLE bill
(
    billno   VARCHAR2(20),
    bdate    DATE,
    bamt     NUMBER(6)
);

CREATE SEQUENCE S3
START WITH 1
INCREMENT BY  1
MAXVALUE 9999;
```

=> use sequence s3 to generate billno for every new bill

```
SQL>INSERT INTO bill
          VALUES('DM/'||TO_CHAR(sysdate,'mmyy')||'/'||S3.NEXTVAL , sysdate,2000);
```

CYCLE option :-

==========

=> by default sequence created with NOCYCLE , i.e. it starts from start with and generates up to max and after generating max then it stops.

=> if sequence created with CYCLE then after reaching max then it will reset to min

```
SQL>CREATE SEQUENCE S10
        START WITH 1
        INCREMENT BY 1
        MAXVALUE 5
```

```
        MINVALUE 1
        CYCLE
        CACHE 4;
```

CACHE size :-
==========

```
SQL>CREATE SEQUENCE S11
        START WITH 1
        INCREMENT BY 1
        MAXVALUE 9999
        MINVALUE 1
        CYCLE
        CACHE 100;
```

CACHE 100:-
==========

=> oracle preallocates next 100 vaues in cache memory and every time we call sequence oracle goes to cache memory and returns sequence next value from cache memory and accessing cache memory is faster than accessing db , so this improves performance.

=> default cache size is 20

=> cache size must be less than 1 cycle

```
SQL>CREATE SEQUENCE S20
        START WITH 1
        INCREMENT BY 1
        MAXVALUE 10
        MINVALUE 1
        CYCLE ;
```

above command fails when executed what could be the reason ?

ans :- default cache size is 20 but it is exceeding cycle (10)

USER_SEQUENCES :-
==========

=> stores information about sequences created by user

SQL>CREATE SEQUENCE S25 ;

=> display sequence s25 details ?

```
SQL>SELECT MIN_VALUE,MAX_VALUE,INCREMENT_BY,CYCLE_FLAG,CACHE_SIZE
      FROM USER_SEQUENCES
      WHERE SEQUENCE_NAME='S25' ;

 MIN_VALUE  MAX_VALUE INCREMENT_BY  C         CACHE_SIZE
---------- ----------  ----------     ---     ----------
 1                    1.0000E+28           1            N        20
```

Droping sequene :-
==========

SQL>DROP SEQUENCE S1 ;

Question :-

SQL> DROP TABLE products;

What is the implication of this command? (Choose all that apply.)

A. All data along with the table structure is deleted. T
B. The pending transaction in the session is committed. T
C. All indexes on the table will remain but they are invalidated.
D. All views and synonyms will remain but they are invalidated. T
E. All data in the table are deleted but the table structure will remain. F

5-nov-26

how to import data from csv file into oracle table :-
==========

step 1 :- create directory object

SYSTEM :-
==========

SQL>CREATE DIRECTORY D10 AS 'D:\NARESH' ;

SQL>GRANT READ,WRITE ON DIRECTORY D10 TO BATCH60 ;

step 2 :- import data

```
    CREATE TABLE  dept55 (
     deptno varchar2(10)  ,
     dname  varchar2(20),
     loc        varchar2(20)
  )
  ORGANIZATION EXTERNAL (
  TYPE ORACLE_LOADER
  DEFAULT DIRECTORY D10
  ACCESS PARAMETERS (
      RECORDS DELIMITED BY NEWLINE
      FIELDS TERMINATED BY ','
      MISSING FIELD VALUES ARE NULL
      (
        deptno,
        dname,
        loc
      )
  )
  LOCATION ('dept.csv')
 )
 REJECT LIMIT UNLIMITED;
```

6-nov-26

INDEXES :-
==========

 => index is also a db object created to improve the performance of data accessing.

 => index makes data accessing faster.

=> index in db is similar to index in textbook , In textbook using index a particular topic can be located fastly ,In db using index a particular record can be located fastly.

 => indexes created on columns and that columns are called index keys

 => indexes created on columns

   1 which are frequently used in where clause
   2 which are used in join operation

Types of indexes :-
==========

1 BTREE indexes
    simple index
    composite index
    unique index
2 BITMAP indexes

simple btree index :-
==========

=> if index created on single column then it is called simple index

CREATE INDEX <name> ON <tabname>(colname);

Ex :-

SQL>CREATE INDEX I1 ON emp(sal) ;

=> when above command is executed oralce creates a structure called BTREE (binatry tree)

EMP 3000
sal
5000
1000 2000 4000
3000
2000 1000 * 2500 * 4000 * 5000 *
1500 1500 * 3000 *,*
4000 2000 *
3000 * => rowid
2500

=> whenever we submit a query , oracle uses following methods to fetch records

1 TABLE SCAN
2 INDEX SCAN

=> In table scan oracle scans complete table i.e. each and every row but in Index scan on avg oracle scans only half of the table , so index scan is faster than table scan.

SELECT * FROM emp WHERE sal = 3000 ; (INDEX SCAN)

SELECT * FROM emp WHERE sal >= 3000; (INDEX SCAN)
SELECT * FROM emp WHERE sal <= 3000; (INDEX SCAN)

SELECT * FROM emp WHERE sal <> 3000; (TABLE SCAN)
SELECT * FROM emp WHERE ename='BLAKE' ; (TABLE SCAN)
SELECT * FROM emp ; (TABLE SCAN)

composite index :-
==========

=> if index created on multiple columns then index is called composite index

ex :- SQL>CREATE INDEX I2 ON EMP(deptno,job) ;

20

10 30

10 CLERK * 20 ANALYST *,* 30 CLERK *
10 MANAGER * 20 CLERK *,* 30 MANAGER *
20 MANAGER * 30 SALESMAN *,*,*,*

=> oracle uses above index when where cond based on leading column of the index i.e. deptno

SELECT * FROM emp WHERE deptno = 20 ; (INDEX)
SELECT * FROM emp WHERE deptno=20 AND job='CLERK' ; (INDEX)
SELECT * FROM emp WHERE job='CLERK'; (TABLE)

UNIQUE index :-
==========

=> unique index doesn't allow duplicate values into the column on which index is created

ex :- SQL>CREATE UNIQUE INDEX I3 ON EMP(ENAME);

K

G Q

ADAMS * JAMES * MARTIN * SCOTT *
ALLEN * JONES * MILLER * SMITH (
BLAKE *

1 SELECT * FROM emp WHERE ename='BLAKE' ;
2 INSERT INTO emp(empno,ename,sal) VALUES(888,'BLAKE',3000); => ERROR

What are different methods to enforce uniqueness ?

1 declare primary key / unique constraint
2 create unique index

=> PRIMARY KEY / UNIQUE columns are implicitly indexed by oracle and
oracle creates a unique index on PRIMARY KEY / UNIQUE columns
and unique index doesn't allow duplicate , so PRIMARY KEY / UNIQUE
also doesn't allow duplicates.

BITMAP indexes :-
==========-

=> bitmap indexes created on low cardinality columns i.e. columns with less distinct values
(< 200)

ex :- gender,state,job,deptno

=> bitmap index stores bits (1 & 0)

BI1(GENDER)		EMP			BI2(JOB)		
M	F	GENDER	JOB		ANALYST	CLERK	MGR
1	0	M	CLERK		0	1	0
0	1	F	MGR	0	0	1	
1	0	M	ANALYST		1	0	0
0	1	F	CLERK		0	1	0
1	0	M	MGR	0	0	1	
1	0	M	ANALYST		1	0	0

SELECT * FROM EMP WHERE GENDER='M' ;